

Maximizing the number of classes in the first room results in having  $\{B, C, F, G\}$  in one room, and classes  $A, D$ , and  $E$  each in their own rooms, for a total of 4. The optimal solution is to put  $A$  in one room,  $\{B, C, D\}$  in another, and  $\{E, F, G\}$  in another, for a total of 3 rooms.

**Problem-9** For [Problem-8](#), consider the following algorithm. Process the classes in increasing order of start times. Assume that we are processing class  $C$ . If there is a room  $R$  such that  $R$  has been assigned to an earlier class, and  $C$  can be assigned to  $R$  without overlapping previously assigned classes, then assign  $C$  to  $R$ . Otherwise, put  $C$  in a new room. Does this algorithm solve the problem?

**Solution:** This algorithm solves the interval-coloring problem. Note that if the greedy algorithm creates a new room for the current class  $c_i$ , then because it examines classes in order of start times,  $c_i$  start point must intersect with the last class in all of the current rooms. Thus when greedy creates the last room,  $n$ , it is because the start time of the current class intersects with  $n - 1$  other classes. But we know that for any single point in any class it can only intersect with at most  $s$  other class, so it must then be that  $n \leq S$ . As  $s$  is a lower bound on the total number needed, and greedy is feasible, it is thus also optimal.

**Note:** For optimal solution refer to [Problem-7](#) and for code refer to [Problem-10](#).

**Problem-10** Suppose we are given two arrays  $Start[1..n]$  and  $Finish[1..n]$  listing the start and finish times of each class. Our task is to choose the largest possible subset  $X \in \{1, 2, \dots, n\}$  so that for any pair  $i, j \in X$ , either  $Start[i] > Finish[j]$  or  $Start[j] > Finish[i]$

**Solution:** Our aim is to finish the first class as early as possible, because that leaves us with the most remaining classes. We scan through the classes in order of finish time, and whenever we encounter a class that doesn't conflict with the latest class so far, then we take that class.

```
int LargestTasks(int Start[], int n, int Finish []) {
    sort Finish[];
    rearrange Start[] to match;
    count = 1;
    X[count] = 1;
    for (i = 2; i < n; i++) {
        if(Start[i] > Finish[X[count]]) {
            count = count + 1;
            X[count] = i;
        }
    }
    return X[1 .. count];
}
```

This algorithm clearly runs in  $O(n \log n)$  time due to sorting.

**Problem-11** Consider the making change problem in the country of India. The input to this problem is an integer  $M$ . The output should be the minimum number of coins to make  $M$  rupees of change. In India, assume the available coins are 1,5,10,20,25,50 rupees. Assume that we have an unlimited number of coins of each type.

**For this problem, does the following algorithm produce the optimal solution or not?** Take as many coins as possible from the highest denominations. So for example, to make change for 234 rupees the greedy algorithm would take four 50 rupee coins, one 25 rupee coin, one 5 rupee coin, and four 1 rupee coins.

**Solution:** The greedy algorithm is not optimal for the problem of making change with the minimum number of coins when the denominations are 1,5,10,20,25, and 50. In order to make 40 rupees, the greedy algorithm would use three coins of 25,10, and 5 rupees. The optimal solution is to use two 20-shilling coins.

**Note:** For the optimal solution, refer to the [Dynamic Programming](#) chapter.

**Problem-12** Let us assume that we are going for a long drive between cities A and B. In preparation for our trip, we have downloaded a map that contains the distances in miles between all the petrol stations on our route. Assume that our car's tanks can hold petrol for  $n$  miles. Assume that the value  $n$  is given. Suppose we stop at every point. Does it give the best solution?

**Solution:** Here the algorithm does not produce optimal solution. Obvious Reason: filling at each petrol station does not produce optimal solution.

**Problem-13** For problem [Problem-12](#), stop if and only if you don't have enough petrol to make it to the next gas station, and if you stop, fill the tank up all the way. Prove or disprove that this algorithm correctly solves the problem.

**Solution:** The greedy approach works: We start our trip from A with a full tank. We check our map to determine the farthest petrol station on our route within  $n$  miles. We stop at that petrol station, fill up our tank and check our map again to determine the farthest petrol station on our route within  $n$  miles from this stop. Repeat the process until we get to B.

**Note:** For code, refer to [Dynamic Programming](#) chapter.

**Problem-14** **Fractional Knapsack problem:** Given items  $t_1, t_2, \dots, t_n$  (items we might want to carry in our backpack) with associated weights  $s_1, s_2, \dots, s_n$  and benefit values  $v_1, v_2, \dots, v_n$ , how can we maximize the total benefit considering that we are subject to an absolute weight limit  $C$ ?

**Solution:**

**Algorithm:**

- 1) Compute value per size density for each item  $d_i = \frac{v_i}{s_i}$ .
- 2) Sort each item by its value density.
- 3) Take as much as possible of the density item not already in the bag

Time Complexity:  $O(n \log n)$  for sorting and  $O(n)$  for greedy selections.

**Note:** The items can be entered into a priority queue and retrieved one by one until either the bag is full or all items have been selected. This actually has a better runtime of  $O(n + c \log n)$  where  $c$  is the number of items that actually get selected in the solution. There is a savings in runtime if  $c = O(n)$ , but otherwise there is no change in the complexity.

**Problem-15 Number of railway-platforms:** At a railway station, we have a time-table with the trains' arrivals and departures. We need to find the minimum number of platforms so that all the trains can be accommodated as per their schedule.

**Example:** The timetable is as given below, the answer is 3. Otherwise, the railway station will not be able to accommodate all the trains.

| Rail   | Arrival  | Departure |
|--------|----------|-----------|
| Rail A | 0900 hrs | 0930 hrs  |
| Rail B | 0915 hrs | 1300 hrs  |
| Rail C | 1030 hrs | 1100 hrs  |
| Rail D | 1045 hrs | 1145 hrs  |

**Solution:** Let's take the same example as described above. Calculating the number of platforms is done by determining the maximum number of trains at the railway station at any time.

First, sort all the arrival( $A$ ) and departure( $D$ ) times in an array. Then, save the corresponding arrivals and departures in the array also. After sorting, our array will look like this:

|      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|
| 0900 | 0915 | 0930 | 1030 | 1045 | 1100 | 1145 | 1300 |
| A    | A    | D    | A    | A    | D    | D    | D    |

Now modify the array by placing 1 for  $A$  and -1 for  $D$ . The new array will look like this:

|   |   |    |   |   |    |    |    |
|---|---|----|---|---|----|----|----|
| 1 | 1 | -1 | 1 | 1 | -1 | -1 | -1 |
|---|---|----|---|---|----|----|----|

Finally make a cumulative array out of this:

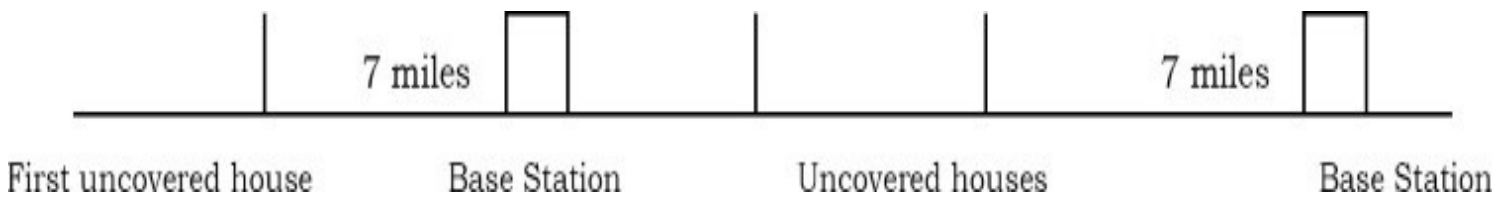
|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 1 | 2 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|

Our solution will be the maximum value in this array. Here it is 3.

**Note:** If we have a train arriving and another departing at the same time, then put the departure time first in the sorted array.

**Problem-16** Consider a country with very long roads and houses along the road. Assume that the residents of all houses use cell phones. We want to place cell phone towers along the road, and each cell phone tower covers a range of 7 kilometers. Create an efficient algorithm that allow for the fewest cell phone towers.

**Solution:**



The algorithm to locate the least number of cell phone towers:

- 1) Start from the beginning of the road
- 2) Find the first uncovered house on the road
- 3) If there is no such house, terminate this algorithm. Otherwise, go to next step
- 4) Locate a cell phone tower 7 miles away after we find this house along the road
- 5) Go to step 2

**Problem-17 Preparing Songs Cassette:** Suppose we have a set of  $n$  songs and want to store these on a tape. In the future, users will want to read those songs from the tape. Reading a song from a tape is not like reading from a disk; first we have to fast-forward past all the other songs, and that takes a significant amount of time. Let  $A[1 .. n]$  be an array listing the lengths of each song, specifically, song  $i$  has length  $A[i]$ . If the songs are stored in order from 1 to  $n$ , then the cost of accessing the  $k^{th}$  song is:

$$C(k) = \sum_{i=1}^k A[i]$$

The cost reflects the fact that before we read song  $k$  we must first scan past all the earlier songs on the tape. If we change the order of the songs on the tape, we change the cost of accessing the songs, with the result that some songs become more expensive to read, but others become cheaper. Different song orders are likely to result in different expected costs. If we assume that each song is equally likely to be accessed, which order should we use if we want the expected cost to be as small as possible?

**Solution:** The answer is simple. We should store the songs in the order from shortest to longest. Storing the short songs at the beginning reduces the forwarding times for the remaining jobs.

**Problem-18** Let us consider a set of events at *HITEX (Hyderabad Convention Center)*. Assume that there are  $n$  events where each takes one unit of time. Event  $i$  will provide a profit of  $P[i]$  rupees ( $P[i] > 0$ ) if started at or before time  $T[i]$ , where  $T[i]$  is an arbitrary number. If an event is not started by  $T[i]$  then there is no benefit in scheduling it at all. All events can start as early as time 0. Give the efficient algorithm to find a schedule that maximizes the profit.

**Solution:**

**Algorithm:**

- Sort the jobs according to  $\text{floor}(T[i])$  (sorted from largest to smallest).
- Let time  $t$  be the current time being considered (where initially  $t = \text{floor}(T[i])$ ).
- All jobs  $i$  where  $\text{floor}(T[i]) = t$  are inserted into a priority queue with the profit  $g_i$  used as the key.
- A *DeleteMax* is performed to select the job to run at time  $t$ .
- Then  $t$  is decremented and the process is continued.

Clearly the time complexity is  $O(n \log n)$ . The sort takes  $O(n \log n)$  and there are at most  $n$  insert and *DeleteMax* operations performed on the priority queue, each of which takes  $O(\log n)$  time.

**Problem-19** Let us consider a customer-care server (say, mobile customer-care) with  $n$  customers to be served in the queue. For simplicity assume that the service time required by each customer is known in advance and it is  $w_i$  minutes for customer  $i$ . So if, for example, the customers are served in order of increasing  $i$ , then the  $i^{\text{th}}$  customer has to wait:  $\sum_{j=1}^{i-1} w_j$  minutes. The total waiting time of all customers can be given as  $= \sum_{i=1}^n \sum_{j=1}^{i-1} w_j$ . What is the best way to serve the customers so that the total waiting time can be reduced?

**Solution:** This problem can be easily solved using greedy technique. Since our objective is to reduce the total waiting time, what we can do is, select the customer whose service time is less. That means, if we process the customers in the increasing order of service time then we can reduce the total waiting time.

Time Complexity:  $O(n \log n)$ .

---

# DIVIDE AND CONQUER ALGORITHMS

CHAPTER

18



## 18.1 Introduction

In the *Greedy* chapter, we have seen that for many problems the Greedy strategy failed to provide optimal solutions. Among those problems, there are some that can be easily solved by using the *Divide and Conquer* (D & C) technique. Divide and Conquer is an important algorithm design technique based on recursion.

The *D & C* algorithm works by recursively breaking down a problem into two or more sub problems of the same type, until they become simple enough to be solved directly. The solutions to the sub problems are then combined to give a solution to the original problem.

## 18.2 What is Divide and Conquer Strategy?

The D & C strategy solves a problem by:

- 1) *Divide*: Breaking the problem into sub problems that are themselves smaller instances of the same type of problem.
- 2) *Recursion*: Recursively solving these sub problems.

- 3) *Conquer*: Appropriately combining their answers.

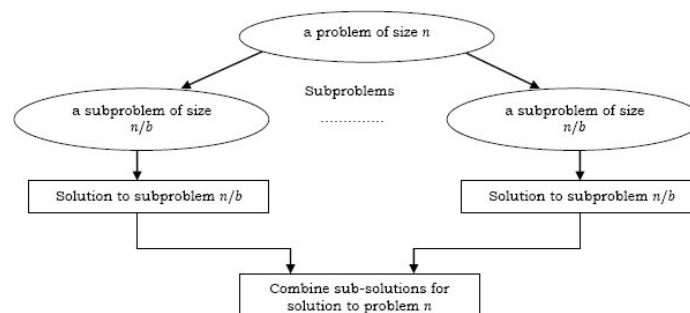
### 18.3 Does Divide and Conquer Always Work?

It's not possible to solve all the problems with the Divide & Conquer technique. As per the definition of D & C, the recursion solves the subproblems which are of the same type. For all problems it is not possible to find the subproblems which are the same size and D & C is not a choice for all problems.

### 18.4 Divide and Conquer Visualization

For better understanding, consider the following visualization. Assume that  $n$  is the size of the original problem. As described above, we can see that the problem is divided into sub problems with each of size  $n/b$  (for some constant  $b$ ). We solve the sub problems recursively and combine their solutions to get the solution for the original problem.

```
DivideAndConquer ( P ) {  
  if( small ( P ) )  
    // P is very small so that a solution is obvious  
    return solution ( n );  
  divide the problem P into k sub problems P1, P2, ..., Pk;  
  return (   
    Combine (   
      DivideAndConquer ( P1 ),  
      DivideAndConquer ( P2 ),  
      ...  
      DivideAndConquer ( Pk )  
    )  
  );  
}
```



## 18.5 Understanding Divide and Conquer

For a clear understanding of *D & C*, let us consider a story. There was an old man who was a rich farmer and had seven sons. He was afraid that when he died, his land and his possessions would be divided among his seven sons, and that they would quarrel with one another.

So he gathered them together and showed them seven sticks that he had tied together and told them that anyone who could break the bundle would inherit everything. They all tried, but no one could break the bundle. Then the old man untied the bundle and broke the sticks one by one. The brothers decided that they should stay together and work together and succeed together. The moral for problem solvers is different. If we can't solve the problem, divide it into parts, and solve one part at a time.

In earlier chapters we have already solved many problems based on *D & C* strategy: like Binary Search, Merge Sort, Quick Sort, etc.... Refer to those topics to get an idea of how *D & C* works. Below are a few other real-time problems which can easily be solved with *D & C* strategy. For all these problems we can find the subproblems which are similar to the original problem.

- Looking for a name in a phone book: We have a phone book with names in alphabetical order. Given a name, how do we find whether that name is there in the phone book or not?
- Breaking a stone into dust: We want to convert a stone into dust (very small stones).
- Finding the exit in a hotel: We are at the end of a very long hotel lobby with a long series of doors, with one door next to us. We are looking for the door that leads to the exit.
- Finding our car in a parking lot.

## 18.6 Advantages of Divide and Conquer

**Solving difficult problems:** *D & C* is a powerful method for solving difficult problems. As an example, consider the Tower of Hanoi problem. This requires breaking the problem into subproblems, solving the trivial cases and combining the subproblems to solve the original problem. Dividing the problem into subproblems so that subproblems can be combined again is a major difficulty in designing a new algorithm. For many such problems *D & C* provides a simple solution.

**Parallelism:** Since *D & C* allows us to solve the subproblems independently, this allows for execution in multiprocessor machines, especially shared-memory systems where the communication of data between processors does not need to be planned in advance, because different subproblems can be executed on different processors.

**Memory access:** *D & C* algorithms naturally tend to make efficient use of memory caches. This is because once a subproblem is small, all its subproblems can be solved within the cache, without



accessing the slower main memory.

## 18.7 Disadvantages of Divide and Conquer

One disadvantage of the  $D \& C$  approach is that recursion is slow. This is because of the overhead of the repeated subproblem calls. Also, the  $D \& C$  approach needs stack for storing the calls (the state at each point in the recursion). Actually this depends upon the implementation style. With large enough recursive base cases, the overhead of recursion can become negligible for many problems.

Another problem with  $D \& C$  is that, for some problems, it may be more complicated than an iterative approach. For example, to add  $n$  numbers, a simple loop to add them up in sequence is much easier than a  $D \& C$  approach that breaks the set of numbers into two halves, adds them recursively, and then adds the sums.

## 18.8 Master Theorem

As stated above, in the  $D \& C$  method, we solve the sub problems recursively. All problems are generally defined in terms of recursive definitions. These recursive problems can easily be solved using Master theorem. For details on Master theorem, refer to the *Introduction to Analysis of Algorithms* chapter. Just for continuity, let us reconsider the Master theorem.

If the recurrence is of the form  $T(n) = aT(\frac{n}{b}) + \Theta(n^k \log^p n)$ , where  $a \geq 1$ ,  $b > 1$ ,  $k \geq 0$  and  $p$  is a real number, then the complexity can be directly given as:

- 1) If  $a > b^k$ , then  $T(n) = \Theta(n^{\log_b a})$
- 2) If  $a = b^k$ 
  - a. If  $p > -1$ , then  $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$
  - b. If  $p = -1$ , then  $T(n) = \Theta(n^{\log_b a} \log \log n)$
  - c. If  $p < -1$ , then  $T(n) = \Theta(n^{\log_b a})$
- 3) If  $a < b^k$ 
  - a. If  $p > 0$ , then  $T(n) = \Theta(n^k \log^p n)$
  - b. If  $p < 0$ , then  $T(n) = O(n^k)$

## 18.9 Divide and Conquer Applications

- Binary Search
- Merge Sort and Quick Sort

- Median Finding
- Min and Max Finding
- Matrix Multiplication
- Closest Pair problem

## 18.10 Divide and Conquer: Problems & Solutions

**Problem-1** Let us consider an algorithm  $A$  which solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time. What is the complexity of this algorithm?

**Solution:** Let us assume that the input size is  $n$  and  $T(n)$  defines the solution to the given problem. As per the description, the algorithm divides the problem into 5 sub problems with each of size  $\frac{n}{2}$ . So we need to solve  $5T(\frac{n}{2})$  subproblems. After solving these sub problems, the given array (linear time) is scanned to combine these solutions. The total recurrence algorithm for this problem can be given as:  $T(n) = 5T(\frac{n}{2}) + O(n)$ . Using the Master theorem (of D & C), we get the complexity  $O(n^{\log_2 5}) \approx O(n^{2+}) \approx O(n^3)$ .

**Problem-2** Similar to [Problem-1](#), an algorithm  $B$  solves problems of size  $n$  by recursively solving two subproblems of size  $n - 1$  and then combining the solutions in constant time. What is the complexity of this algorithm?

**Solution:** Let us assume that the input size is  $n$  and  $T(n)$  defines the solution to the given problem. As per the description of algorithm we divide the problem into 2 sub problems with each of size  $n - 1$ . So we have to solve  $2T(n - 1)$  sub problems. After solving these sub problems, the algorithm takes only a constant time to combine these solutions. The total recurrence algorithm for this problem can be given as:

$$T(n) = 2T(n - 1) + O(1)$$

Using Master theorem (of *Subtract and Conquer*), we get the complexity as  $O(n^0 2^{\frac{n}{1}}) = O(2^n)$ . (Refer to [Introduction](#) chapter for more details).

**Problem-3** Again similar to [Problem-1](#), another algorithm  $C$  solves problems of size  $n$  by dividing them into nine subproblems of size  $\frac{n}{3}$ , recursively solving each subproblem, and then combining the solutions in  $O(n^2)$  time. What is the complexity of this algorithm?

**Solution:** Let us assume that input size is  $n$  and  $T(n)$  defines the solution to the given problem. As per the description of algorithm we divide the problem into 9 sub problems with each of size  $\frac{n}{3}$ . So we need to solve  $9T(\frac{n}{3})$  sub problems. After solving the sub problems, the algorithm takes

quadratic time to combine these solutions. The total recurrence algorithm for this problem can be given as:  $T(n) = 9T\left(\frac{n}{3}\right) + O(n^2)$ . Using D & C Master theorem, we get the complexity as  $O(n^2 \log n)$ .

**Problem-4** Write a recurrence and solve it.

```
void function(n) {
    if(n > 1) {
        printf("*");
        function( $\frac{n}{2}$ );
        function( $\frac{n}{2}$ );
    }
}
```

**Solution:** Let us assume that input size is  $n$  and  $T(n)$  defines the solution to the given problem. As per the given code, after printing the character and dividing the problem into 2 subproblems with each of size  $\frac{n}{2}$  and solving them. So we need to solve  $2T\left(\frac{n}{2}\right)$  subproblems. After solving these subproblems, the algorithm is not doing anything for combining the solutions. The total recurrence algorithm for this problem can be given as:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

Using Master theorem (of D & C), we get the complexity as  $O(n^{\log_2 2}) \approx O(n^1) = O(n)$ .

**Problem-5** Given an array, give an algorithm for finding the maximum and minimum.

**Solution:** Refer [Selection Algorithms](#) chapter.

**Problem-6** Discuss Binary Search and its complexity.

**Solution:** Refer [Searching](#) chapter for discussion on Binary Search.

**Analysis:** Let us assume that input size is  $n$  and  $T(n)$  defines the solution to the given problem. The elements are in sorted order. In binary search we take the middle element and check whether the element to be searched is equal to that element or not. If it is equal then we return that element.

If the element to be searched is greater than the middle element then we consider the right sub-array for finding the element and discard the left sub-array. Similarly, if the element to be searched is less than the middle element then we consider the left sub-array for finding the element and discard the right sub-array.

What this means is, in both the cases we are discarding half of the sub-array and considering the

remaining half only. Also, at every iteration we are dividing the elements into two equal halves.

As per the above discussion every time we divide the problem into 2 sub problems with each of size  $\frac{n}{2}$  and solve one  $T\left(\frac{n}{2}\right)$  sub problem. The total recurrence algorithm for this problem can be given as:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

Using Master theorem (of D & C), we get the complexity as  $O(\log n)$ .

**Problem-7** Consider the modified version of binary search. Let us assume that the array is divided into 3 equal parts (ternary search) instead of 2 equal parts. Write the recurrence for this ternary search and find its complexity.

**Solution:** From the discussion on [Problem-5](#), binary search has the recurrence relation:  $T(n) = T\left(\frac{n}{2}\right) + O(1)$ . Similar to the [Problem-5](#) discussion, instead of 2 in the recurrence relation we use “3”. That indicates that we are dividing the array into 3 sub-arrays with equal size and considering only one of them. So, the recurrence for the ternary search can be given as:

$$T(n) = T\left(\frac{n}{3}\right) + O(1)$$

Using Master theorem (of D & C), we get the complexity as  $O(\log_3 n) \approx O(\log n) \approx O(\log n)$  (we don't have to worry about the base of  $\log$  as they are constants).

**Problem-8** In [Problem-5](#), what if we divide the array into two sets of sizes approximately one-third and two-thirds.

**Solution:** We now consider a slightly modified version of ternary search in which only one comparison is made, which creates two partitions, one of roughly  $\frac{n}{3}$  elements and the other of  $\frac{2n}{3}$ . Here the worst case comes when the recursive call is on the larger  $\frac{2n}{3}$  element part. So the recurrence corresponding to this worst case is:

$$T(n) = T\left(\frac{2n}{3}\right) + O(1)$$

Using Master theorem (of D & C), we get the complexity as  $O(\log n)$ . It is interesting to note that we will get the same results for general  $k$ -ary search (as long as  $k$  is a fixed constant which does not depend on  $n$ ) as  $n$  approaches infinity.

**Problem-9** Discuss Merge Sort and its complexity.

**Solution:** Refer to [Sorting](#) chapter for discussion on Merge Sort. In Merge Sort, if the number of elements are greater than 1, then divide them into two equal subsets, the algorithm is recursively

invoked on the subsets, and the returned sorted subsets are merged to provide a sorted list of the original set. The recurrence equation of the Merge Sort algorithm is:

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + O(n), & \text{if } n > 1 \\ 0 & , \text{if } n = 1 \end{cases}$$

If we solve this recurrence using D & C Master theorem it gives  $O(n \log n)$  complexity.

**Problem-10** Discuss Quick Sort and its complexity.

**Solution:** Refer to [Sorting](#) chapter for discussion on Quick Sort. For Quick Sort we have different complexities for best case and worst case.

**Best Case:** In *Quick Sort*, if the number of elements is greater than 1 then they are divided into two equal subsets, and the algorithm is recursively invoked on the subsets. After solving the sub problems we don't need to combine them. This is because in *Quick Sort* they are already in sorted order. But, we need to scan the complete elements to partition the elements. The recurrence equation of *Quick Sort* best case is

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + O(n), & \text{if } n > 1 \\ 0 & , \text{if } n = 1 \end{cases}$$

If we solve this recurrence using Master theorem of D & C gives  $O(n \log n)$  complexity.

**Worst Case:** In the worst case, Quick Sort divides the input elements into two sets and one of them contains only one element. That means other set has  $n - 1$  elements to be sorted. Let us assume that the input size is  $n$  and  $T(n)$  defines the solution to the given problem. So we need to solve  $T(n - 1)$ ,  $T(1)$  subproblems. But to divide the input into two sets Quick Sort needs one scan of the input elements (this takes  $O(n)$ ).

After solving these sub problems the algorithm takes only a constant time to combine these solutions. The total recurrence algorithm for this problem can be given as:

$$T(n) = T(n - 1) + O(1) + O(n).$$

This is clearly a summation recurrence equation. So,  $T(n) = \frac{n(n+1)}{2} = O(n^2)$ .

**Note:** For the average case analysis, refer to [Sorting](#) chapter.

**Problem-11** Given an infinite array in which the first  $n$  cells contain integers in sorted order and the rest of the cells are filled with some special symbol (say, \$). Assume we do not know the  $n$  value. Give an algorithm that takes an integer  $K$  as input and finds a position in the array containing  $K$ , if such a position exists, in  $O(\log n)$  time.

**Solution:** Since we need an  $O(\log n)$  algorithm, we should not search for all the elements of the given list (which gives  $O(n)$  complexity). To get  $O(\log n)$  complexity one possibility is to use binary search. But in the given scenario we cannot use binary search as we do not know the end of the list. Our first problem is to find the end of the list. To do that, we can start at the first element and keep searching with doubled index. That means we first search at index 1 then, 2, 4, 8 ...

```
int FindInInfiniteSeries(int A[]) {
    int mid, l = r = 1;
    while( A[r] != '$') {
        l = r;
        r = r * 2;
    }
    while( (r - l > 1) ) {
        mid = (r + l) / 2;
        if( A[mid] == '$')
            r = mid;
        else
            l = mid;
    }
}
```

It is clear that, once we have identified a possible interval  $A[i, \dots, 2i]$  in which  $K$  might be, its length is at most  $n$  (since we have only  $n$  numbers in the array  $A$ ), so searching for  $K$  using binary search takes  $O(\log n)$  time.

**Problem-12** Given a sorted array of non-repeated integers  $A[1.. n]$ , check whether there is an index  $i$  for which  $A[i] = i$ . Give a divide-and-conquer algorithm that runs in time  $O(\log n)$ .

**Solution:** We can't use binary search on the array as it is. If we want to keep the  $O(\log n)$  property of the solution we have to implement our own binary search. If we modify the array (in place or in a copy) and subtract  $i$  from  $A[i]$ , we can then use binary search. The complexity for doing so is  $O(n)$ .

**Problem-13** We are given two sorted lists of size  $n$ . Give an algorithm for finding the median element in the union of the two lists.

**Solution:** We use the Merge Sort process. Use *merge* procedure of merge sort (refer to [Sorting](#) chapter). Keep track of the count while comparing elements of two arrays. If the count becomes  $n$  (since there are  $2n$  elements), we have reached the median. Take the average of the elements at indexes  $n - 1$  and  $n$  in the merged array.

Time Complexity:  $O(n)$ .

**Problem-14** Can we give the algorithm if the size of the two lists are not the same?



**Solution:** The solution is similar to the previous problem. Let us assume that the lengths of two lists are  $m$  and  $n$ . In this case we need to stop when the counter reaches  $(m + n)/2$ .

Time Complexity:  $O((m + n)/2)$ .

**Problem-15** Can we improve the time complexity of [Problem-13](#) to  $O(\log n)$ ?

**Solution:** Yes, using the D & C approach. Let us assume that the given two lists are  $L1$  and  $L2$ .

**Algorithm:**

1. Find the medians of the given sorted input arrays  $L1[]$  and  $L2[]$ . Assume that those medians are  $m1$  and  $m2$ .
2. If  $m1$  and  $m2$  are equal then return  $m1$  (or  $m2$ ).
3. If  $m1$  is greater than  $m2$ , then the final median will be below two sub arrays.
4. From first element of  $L1$  to  $m1$ .
5. From  $m2$  to last element of  $L2$ .
6. If  $m2$  is greater than  $m1$ , then median is present in one of the two sub arrays below.
7. From  $m1$  to last element of  $L1$ .
8. From first element of  $L2$  to  $m2$ .
9. Repeat the above process until the size of both the sub arrays becomes 2.
10. If size of the two arrays is 2, then use the formula below to get the median.
11. Median =  $(\max(L1[0], L2[0]) + \min(L1[1], L2[1]))/2$

Time Complexity:  $O(\log n)$  since we are considering only half of the input and throwing the remaining half.

**Problem-16** Given an input array  $A$ . Let us assume that there can be duplicates in the list. Now search for an element in the list in such a way that we get the highest index if there are duplicates.

**Solution:** Refer to [Searching](#) chapter.

**Problem-17** Discuss Strassen's Matrix Multiplication Algorithm using Divide and Conquer. That means, given two  $n \times n$  matrices,  $A$  and  $B$ , compute the  $n \times n$  matrix  $C = A \times B$ , where the elements of  $C$  are given by

$$C_{i,j} = \sum_{k=0}^{n-1} A_{i,k} B_{k,j}$$

**Solution:** Before Strassen's algorithm, first let us see the basic divide and conquer algorithm. The general approach we follow for solving this problem is given below. To determine,  $C[i,j]$  we need to multiply the  $i^{th}$  row of  $A$  with  $j^{th}$  column of  $B$ .

```
// Initialize C.
for i = 1 to n
  for j = 1 to n
    for k = 1 to n
      C[i, j] += A[i, k] * B[k, j];
```

The matrix multiplication problem can be solved with the D & C technique. To implement a D & C algorithm we need to break the given problem into several subproblems that are similar to the original one. In this instance we view each of the  $n \times n$  matrices as a  $2 \times 2$  matrix, the elements of which are  $\frac{n}{2} \times \frac{n}{2}$  submatrices. So, the original matrix multiplication,  $C = A \times B$  can be written as:

$$\begin{bmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{bmatrix} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix} \times \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{bmatrix}$$

where each  $A_{i,j}$ ,  $B_{i,j}$ , and  $C_{i,j}$  is a  $\frac{n}{2} \times \frac{n}{2}$  matrix.

From the given definition of  $C_{i,j}$ , we get that the result sub matrices can be computed as follows:

$$\begin{aligned} C_{1,1} &= A_{1,1} \times B_{1,1} + A_{1,2} \times B_{2,1} \\ C_{1,2} &= A_{1,1} \times B_{1,2} + A_{1,2} \times B_{2,2} \\ C_{2,1} &= A_{2,1} \times B_{1,1} + A_{2,2} \times B_{2,1} \\ C_{2,2} &= A_{2,1} \times B_{1,2} + A_{2,2} \times B_{2,2} \end{aligned}$$

Here the symbols  $+$  and  $\times$  are taken to mean addition and multiplication (respectively) of  $\frac{n}{2} \times \frac{n}{2}$  matrices.

In order to compute the original  $n \times n$  matrix multiplication we must compute eight  $\frac{n}{2} \times \frac{n}{2}$  matrix products (*divide*) followed by four  $\frac{n}{2} \times \frac{n}{2}$  matrix sums (*conquer*). Since matrix addition is an  $O(n^2)$  operation, the total running time for the multiplication operation is given by the recurrence:

$$T(n) = \begin{cases} O(1) & , for\ n = 1 \\ 8T\left(\frac{n}{2}\right) + O(n^2) & , for\ n > 1 \end{cases}$$

Using master theorem, we get  $T(n) = O(n^3)$ .

Fortunately, it turns out that one of the eight matrix multiplications is redundant (found by Strassen). Consider the following series of seven  $\frac{n}{2} \times \frac{n}{2}$  matrices:



$$\begin{aligned}
M_0 &= (A_{1,1} + A_{2,2}) \times (B_{1,1} + B_{2,2}) \\
M_1 &= (A_{1,2} - A_{2,2}) \times (B_{2,1} + B_{2,2}) \\
M_2 &= (A_{1,1} - A_{2,1}) \times (B_{1,1} + B_{1,2}) \\
M_3 &= (A_{1,1} + A_{1,2}) \times B_{2,2} \\
M_4 &= A_{1,1} \times (B_{1,2} - B_{2,2}) \\
M_5 &= A_{2,2} \times (B_{2,1} - B_{1,1}) \\
M_6 &= (A_{2,1} + A_{2,2}) \times B_{1,1}
\end{aligned}$$

Each equation above has only one multiplication. Ten additions and seven multiplications are required to compute  $M_0$  through  $M_6$ . Given  $M_0$  through  $M_6$ , we can compute the elements of the product matrix  $C$  as follows:

$$\begin{aligned}
C_{1,1} &= M_0 + M_1 - M_3 + M_5 \\
C_{1,2} &= M_3 + M_4 \\
C_{2,1} &= M_5 + M_6 \\
C_{2,2} &= M_0 - M_2 + M_4 - M_6
\end{aligned}$$

This approach requires seven  $\frac{n}{2} \times \frac{n}{2}$  matrix multiplications and  $18 \frac{n}{2} \times \frac{n}{2}$  additions. Therefore, the worst-case running time is given by the following recurrence:

$$T(n) = \begin{cases} O(1) & , for\ n = 1 \\ 7T\left(\frac{n}{2}\right) + O(n^2) & , for\ n = 1 \end{cases}$$

Using master theorem, we get,  $T(n) = O(n^{\log_2 7}) = O(n^{2.81})$ .

**Problem-18 Stock Pricing Problem:** Consider the stock price of [CareerMonk.com](https://www.careermonk.com) in  $n$  consecutive days. That means the input consists of an array with stock prices of the company. We know that the stock price will not be the same on all the days. In the input stock prices there may be dates where the stock is high when we can sell the current holdings, and there may be days when we can buy the stock. Now our problem is to find the day on which we can buy the stock and the day on which we can sell the stock so that we can make maximum profit.

**Solution:** As given in the problem, let us assume that the input is an array with stock prices [integers]. Let us say the given array is  $A[1], \dots, A[n]$ . From this array we have to find two days [one for buy and one for sell] in such a way that we can make maximum profit. Also, another point to make is that the buy date should be before sell date. One simple approach is to look at all possible buy and sell dates.

```

void StockStrategy(int A[], int n, int *buyDateIndex, int *sellDateIndex) {
    int j, profit=0;
    *buyDateIndex =0; *sellDateIndex =0;
    for (int i = 1; i < n; i++) //indicates buy date
        //indicates sell date
        for(j = i; j < n; j++)
            if(A[j] - A[i] > profit) {
                profit = A[j] - A[i];
                *buyDateIndex = i;
                *sellDateIndex = j;
            }
    }
}

```

The two nested loops take  $n(n + 1)/2$  computations, so this takes time  $\Theta(n^2)$ .

**Problem-19** For [Problem-18](#), can we improve the time complexity?

**Solution: Yes**, by opting for the Divide-and-Conquer  $\Theta(n \log n)$  solution. Divide the input list into two parts and recursively find the solution in both the parts. Here, we get three cases:

- *buyDateIndex* and *sellDateIndex* both are in the earlier time period.
- *buyDateIndex* and *sellDateIndex* both are in the later time period.
- *buyDateIndex* is in the earlier part and *sellDateIndex* is in the later part of the time period.

The first two cases can be solved with recursion. The third case needs care. This is because *buyDateIndex* is one side and *sellDateIndex* is on other side. In this case we need to find the minimum and maximum prices in the two sub-parts and this we can solve in linear-time.

```

void StockStrategy(int A[], int left, int right) {
    //Declare the necessary variables;
    if(left + 1 = right)
        return (0, left, left);
    mid = left + (right - left) / 2;
    (profitLeft, buyDateIndexLeft, sellDateIndexLeft) = StockStrategy(A, left, mid);
    (profitRight, buyDateIndexRight, sellDateIndexRight) = StockStrategy(A, mid, right);
    minLeft = Min(A, left, mid);
    maxRight = Max(A, mid, right);
    profit = A[maxRight] - A[minLeft];
    if(profitLeft > max(profitRight, profit))
        return (profitLeft, buyDateIndexLeft, sellDateIndexLeft);
    else if(profitRight > max(profitLeft, profit))
        return (profitRight, buyDateIndexRight, sellDateIndexRight);
    else
        return (profit, minLeft, maxRight);
}

```

Algorithm *StockStrategy* is used recursively on two problems of half the size of the input, and in addition  $\Theta(n)$  time is spent searching for the maximum and minimum prices. So the time complexity is characterized by the recurrence  $T(n) = 2T(n/2) + \Theta(n)$  and by the Master theorem we get  $O(n \log n)$ .

**Problem-20** We are testing “unbreakable” laptops and our goal is to find out how unbreakable they really are. In particular, we work in an  $n$ -story building and want to find out the lowest floor from which we can drop the laptop without breaking it (call this “the ceiling”). Suppose we are given two laptops and want to find the highest ceiling possible. Give an algorithm that minimizes the number of tries we need to make  $f(n)$  (hopefully,  $f(n)$  is sub-linear, as a linear  $f(n)$  yields a trivial solution).

**Solution:** For the given problem, we cannot use binary search as we cannot divide the problem and solve it recursively. Let us take an example for understanding the scenario. Let us say 14 is the answer. That means we need 14 drops to find the answer. First we drop from height 14, and if it breaks we try all floors from 1 to 13. If it doesn't break then we are left 13 drops, so we will drop it from  $14 + 13 + 1 = 28^{th}$  floor. The reason being if it breaks at the  $28^{th}$  floor we can try all the floors from 15 to 27 in 12 drops (total of 14 drops). If it did not break, then we are left with 11 drops and we can try to figure out the floor in 14 drops.

From the above example, it can be seen that we first tried with a gap of 14 floors, and then followed by 13 floors, then 12 and so on. So if the answer is  $k$  then we are trying the intervals at  $k, k - 1, k - 2 \dots 1$ . Given that the number of floors is  $n$ , we have to relate these two. Since the maximum floor from which we can try is  $n$ , the total skips should be less than  $n$ . This gives:

$$\begin{aligned}
 k + (k - 1) + (k - 2) + \dots + 1 &\leq n \\
 \frac{k(k + 1)}{2} &\leq n \\
 k &\leq \sqrt{n}
 \end{aligned}$$

Complexity of this process is  $O(\sqrt{n})$ .

**Problem-21** Given  $n$  numbers, check if any two are equal.

**Solution:** Refer to [Searching](#) chapter.

**Problem-22** Give an algorithm to find out if an integer is a square? E.g. 16 is, 15 isn't.

**Solution:** Initially let us say  $i = 2$ . Compute the value  $i \times i$  and see if it is equal to the given number. If it is equal then we are done; otherwise increment the  $i$  value. Continue this process until we reach  $i \times i$  greater than or equal to the given number.

Time Complexity:  $O(\sqrt{n})$ . Space Complexity:  $O(1)$ .

**Problem-23** Given an array of  $2n$  integers in the following format  $a_1 a_2 a_3 \dots a_n b_1 b_2 b_3 \dots b_n$ . Shuffle the array to  $a_1 b_1 a_2 b_2 a_3 b_3 \dots a_n b_n$  without any extra memory [MA].

**Solution:** Let us take an example (for brute force solution refer to [Searching](#) chapter)

1. Start with the array:  $a_1 a_2 a_3 a_4 b_1 b_2 b_3 b_4$
2. Split the array into two halves:  $a_1 a_2 a_3 a_4 : b_1 b_2 b_3 b_4$
3. Exchange elements around the center: exchange  $a_3 a_4$  with  $b_1 b_2$  you get:  $a_1 a_2 b_1 b_2 a_3 a_4 b_3 b_4$
4. Split  $a_1 a_2 b_1 b_2$  into  $a_1 a_2 : b_1 b_2$  then split  $a_3 a_4 b_3 b_4$  into  $a_3 a_4 : b_3 b_4$
5. Exchange elements around the center for each subarray you get:  $a_1 b_1 a_2 b_2$  and  $a_3 b_3 a_4 b_4$

Please note that this solution only handles the case when  $n = 2^i$  where  $i = 0, 1, 2, 3$ , etc. In our example  $n = 2^2 = 4$  which makes it easy to recursively split the array into two halves. The basic idea behind swapping elements around the center before calling the recursive function is to produce smaller size problems. A solution with linear time complexity may be achieved if the elements are of a specific nature. For example you can calculate the new position of the element using the value of the element itself. This is a hashing technique.

```

void ShuffleArray(int A[], int l, int r) {
    //Array center
    int c = l + (r-l)/2, q = 1 + l + (c-l)/2;
    if(l == r) //Base case when the array has only one element
        return;
    for (int k = 1, i = q; i <= c; i++, k++) {
        //Swap elements around the center
        int tmp = A[i];  A[i] = A[c + k];  A[c + k] = tmp;
    }
    ShuffleArray(A, l, c);           //Recursively call the function on the left and right
    ShuffleArray(A, c + 1, r);       //Recursively call the function on the right
}

```

Time Complexity:  $O(n \log n)$ .

**Problem-24 Nuts and Bolts Problem:** Given a set of  $n$  nuts of different sizes and  $n$  bolts such that there is a one-to-one correspondence between the nuts and the bolts, find for each nut its corresponding bolt. Assume that we can only compare nuts to bolts (cannot compare nuts to nuts and bolts to bolts).

**Solution:** Refer to [Sorting](#) chapter.

**Problem-25 Maximum Value Contiguous Subsequence:** Given a sequence of  $n$  numbers  $A(1) \dots A(n)$ , give an algorithm for finding a contiguous subsequence  $A(i) \dots A(j)$  for which the sum of elements in the subsequence is maximum. **Example :**  $\{-2, 11, -4, 13, -5, 2\} \rightarrow 20$  and  $\{1, -3, 4, -2, -1, 6\} \rightarrow 7$ .

**Solution:** Divide this input into two halves. The maximum contiguous subsequence sum can occur in one of 3 ways:

- Case 1: It can be completely in the first half
- Case 2: It can be completely in the second half
- Case 3: It begins in the first half and ends in the second half

We begin by looking at case 3. To avoid the nested loop that results from considering all  $n/2$  starting points and  $n/2$  ending points independently, replace two nested loops with two consecutive loops. The consecutive loops, each of size  $n/2$ , combine to require only linear work. Any contiguous subsequence that begins in the first half and ends in the second half must include both the last element of the first half and the first element of the second half. What we can do in cases 1 and 2 is apply the same strategy of dividing into more halves. In summary, we do the following:

1. Recursively compute the maximum contiguous subsequence that resides entirely in the first half.
2. Recursively compute the maximum contiguous subsequence that resides entirely in the

- second half.
3. Compute, via two consecutive loops, the maximum contiguous subsequence sum that begins in the first half but ends in the second half.
  4. Choose the largest of the three sums.

```
int MaxSumRec(int A[], int left, int right) {
    int MaxLeftBorderSum = 0, MaxRightBorderSum = 0, LeftBorderSum = 0, RightBorderSum = 0;
    int mid = left + (right - left) / 2;
    if(left == right) // Base Case
        return A[left] > 0 ? A[left] : 0;
    int MaxLeftSum = MaxSumRec(A, left, mid);
    int MaxRightSum = MaxSumRec(A, mid + 1, right);
    for (int i = mid; i >= left; i--) {
        LeftBorderSum += A[i];
        if(LeftBorderSum > MaxLeftBorderSum)
            MaxLeftBorderSum = LeftBorderSum;
    }
    for (int j = mid + 1; j <= right; j++) {
        RightBorderSum += A[j];
        if(RightBorderSum > MaxRightBorderSum)
            MaxRightBorderSum = RightBorderSum;
    }
    return Max(MaxLeftSum, MaxRightSum, MaxLeftBorderSum + MaxRightBorderSum);
}

int MaxSubsequenceSum(int A, int n) {
    return n > 0 ? MaxSumRec(A, 0, n - 1) : 0;
}
```

The base case cost is 1. The program performs two recursive calls plus the linear work involved in computing the maximum sum for case 3. The recurrence relation is:

$$T(1) = 1$$

$$T(n) = 2T(n/2) + n$$

Using *D & C* Master theorem, we get the time complexity as  $T(n) = O(n \log n)$ .

**Note:** For an efficient solution refer to the [Dynamic Programming](#) chapter.

**Problem-26      Closest-Pair of Points:** Given a set of  $n$  points,  $S = \{p_1, p_2, p_3, \dots, p_n\}$ , where  $p_i = (x_i, y_i)$ . Find the pair of points having the smallest distance among all pairs (assume that all points are in one dimension).

**Solution:** Let us assume that we have sorted the points. Since the points are in one dimension, all the points are in a line after we sort them (either on X-axis or Y-axis). The complexity of sorting is  $O(n \log n)$ . After sorting we can go through them to find the consecutive points with the least difference. So the problem in one dimension is solved in  $O(n \log n)$  time which is mainly dominated by sorting time.

Time Complexity:  $O(n \log n)$ .

**Problem-27** For [Problem-26](#), how do we solve it if the points are in two-dimensional space?

**Solution:** Before going to the algorithm, let us consider the following mathematical equation:

$$\text{distance}(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

The above equation calculates the distance between two points  $p_1 = (x_1, y_1)$  and  $p_2 = (x_2, y_2)$ .

**Brute Force Solution:**

- Calculate the distances between all the pairs of points. From  $n$  points there are  $n_{c_2}$  ways of selecting 2 points. ( $n_{c_2} = O(n^2)$ ).
- After finding distances for all  $n^2$  possibilities, we select the one which is giving the minimum distance and this takes  $O(n^2)$ .

The overall time complexity is  $O(n^2)$ .

**Problem-28** Give  $O(n \log n)$  solution for *closest pair* problem (Problem-27)?

**Solution:** To find  $O(n \log n)$  solution, we can use the D & C technique. Before starting the divide-and-conquer process let us assume that the points are sorted by increasing x-coordinate. Divide the points into two equal halves based on median of x-coordinates. That means the problem is divided into that of finding the closest pair in each of the two halves. For simplicity let us consider the following algorithm to understand the process.

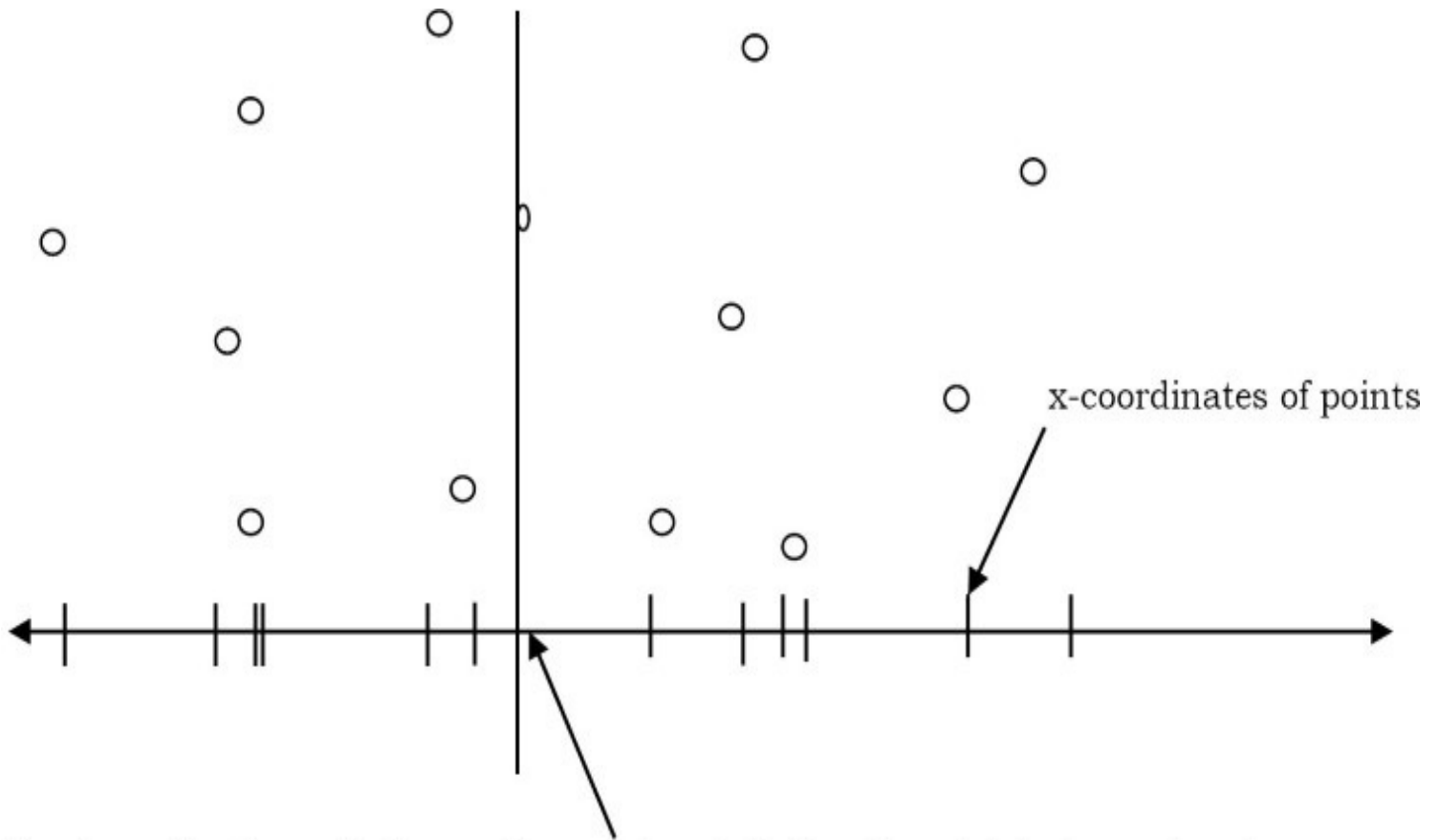
**Algorithm:**

- 1) Sort the given points in  $S$  (given set of points) based on their  $x$  –coordinates. Partition  $S$  into two subsets,  $S_1$  and  $S_2$ , about the line  $l$  through median of  $S$ . This step is the *Divide* part of the *D & C* technique.
- 2) Find the closest-pairs in  $S_1$  and  $S_2$  and call them  $L$  and  $R$  recursively.
- 3) Now, steps 4 to 8 form the Combining component of the *D & C* technique.
- 4) Let us assume that  $\delta = \min(L, R)$ .
- 5) Eliminate points that are farther than  $\delta$  apart from  $l$ .
- 6) Consider the remaining points and sort based on their  $y$ -coordinates.
- 7) Scan the remaining points in the  $y$  order and compute the distances of each point to all its neighbors that are distanced no more than  $2 \times \delta$  (that's the reason for sorting



according to  $y$ ).

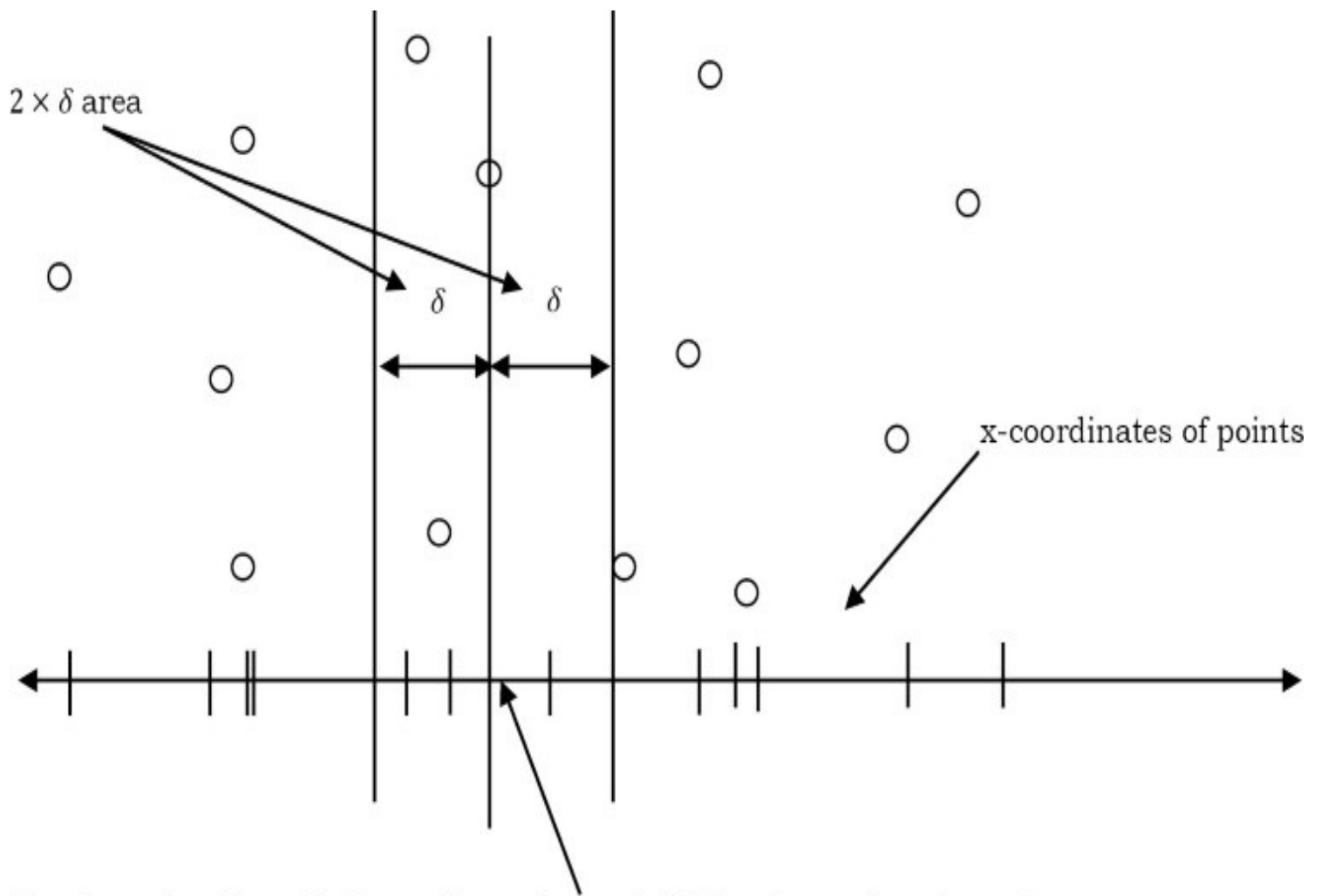
- 8) If any of these distances is less than  $\delta$  then update  $\delta$ .



Line  $l$  passing through the median point and divides the set into 2 equal parts

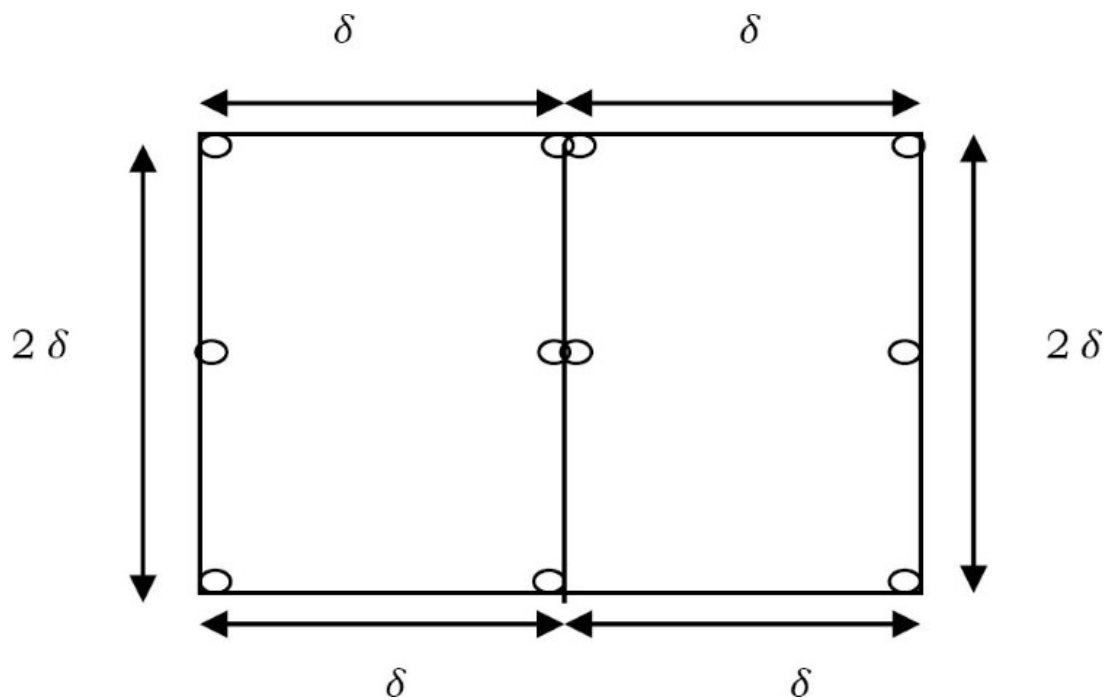
**Combining the results in linear time**





Line  $l$  passing through the median point and divides the set into 2 equal parts

Let  $\delta = \min(L, R)$ , where  $L$  is the solution to first sub problem and  $R$  is the solution to second sub problem. The possible candidates for closest-pair, which are across the dividing line, are those which are less than  $\delta$  distance from the line. So we need only the points which are inside the  $2 \times \delta$  area across the dividing line as shown in the figure. Now, to check all points within distance  $\delta$  from the line, consider the following figure.



From the above diagram we can see that a maximum of 12 points can be placed inside the square with a distance not less than  $\delta$ . That means, we need to check only the distances which are within 11 positions in the sorted list. This is similar to the one above, but with the difference that in the above combining of subproblems, there are no vertical bounds. So we can apply the 12-point box tactic over all the possible boxes in the  $2 \times \delta$  area with the dividing line as the middle line. As there can be a maximum of  $n$  such boxes in the area, the total time for finding the closest pair in the corridor is  $O(n)$ .

### Analysis:

- 1) Step-1 and Step-2 take  $O(n \log n)$  for sorting and recursively finding the minimum.
- 2) Step-4 takes  $O(1)$ .
- 3) Step-5 takes  $O(n)$  for scanning and eliminating.
- 4) Step-6 takes  $O(n \log n)$  for sorting.
- 5) Step-7 takes  $O(n)$  for scanning.

The total complexity:  $T(n) = O(n \log n) + O(1) + O(n) + O(n) + O(n) \approx O(n \log n)$ .

**Problem-29** To calculate  $k^n$ , give algorithm and discuss its complexity.

**Solution:** The naive algorithm to compute  $k^n$  is: start with 1 and multiply by  $k$  until reaching  $k^n$ . For this approach; there are  $n - 1$  multiplications and each takes constant time giving a  $\Theta(n)$  algorithm.

But there is a faster way to compute  $k^n$ . For example,

$$9^{24} = (9^{12})^2 = ((9^6)^2)^2 = (((9^3)^2)^2)^2 = (((9^2 \cdot 9)^2)^2)^2$$

Note that taking the square of a number needs only one multiplication; this way, to compute  $9^{24}$  we need only 5 multiplications instead of 23.

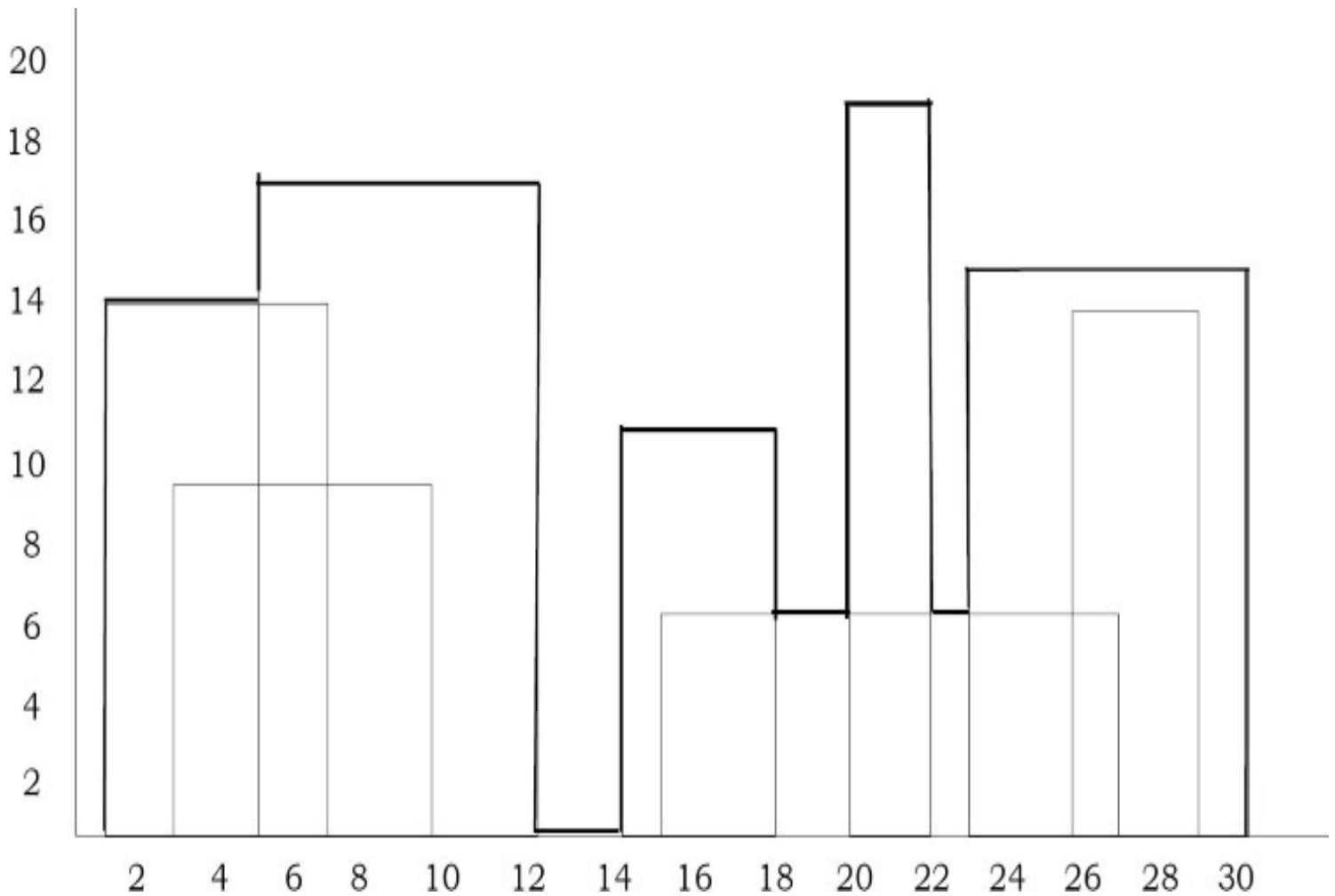
```
int Exponential(int k, int n) {
    if (k == 0)
        return 1;
    else{
        if (n%2 == 1){
            a = Exponential(k, n-1);
            return a*k;
        }
        else{
            a = Exponential(k, n/2);
            return a*a;
        }
    }
}
```

Let  $T(n)$  be the number of multiplications required to compute  $k^n$ . For simplicity, assume  $k = 2^i$  for some  $i \geq 1$ .

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Using master theorem we get  $T(n) = O(\log n)$ .

**Problem-30 The Skyline Problem:** Given the exact locations and shapes of  $n$  rectangular buildings in a 2-dimensional city. There is no particular order for these rectangular buildings. Assume that the bottom of all buildings lie on a fixed horizontal line (bottom edges are collinear). The input is a list of triples; one per building. A building  $B_i$  is represented by the triple  $(l_i, h_i, r_i)$  where  $l_i$  denote the  $x$ -position of the left edge and  $r_i$  denote the  $x$ -position of the right edge, and  $h_i$  denotes the building's height. Give an algorithm that computes the skyline (in 2 dimensions) of these buildings, eliminating hidden lines. In the diagram below there are 8 buildings, represented from left to right by the triplets (1, 14, 7), (3, 9, 10), (5, 17, 12), (14, 11, 18), (15, 6, 27), (20, 19, 22), (23, 15, 30) and (26, 14, 29).



The output is a collection of points which describe the path of the skyline. In some versions of the problem this collection of points is represented by a sequence of numbers  $p_1, p_2, \dots, p_n$ , such that the point  $p_i$  represents a horizontal line drawn at height  $p_i$  if  $i$  is even, and it represents a vertical line drawn at position  $p_i$  if  $i$  is odd. In our case the collection of points will be a sequence of  $p_1, p_2, \dots, p_n$  pairs of  $(x_i, h_i)$  where  $p_i(x_i, h_i)$  represents the  $h_i$  height of the skyline at position  $x_i$ . In the diagram above the skyline is drawn with a thick line around the buildings and it is represented by the sequence of position-height pairs  $(1, 14), (5, 17), (12, 0), (14, 11), (18, 6), (20, 19), (22, 6), (23, 15)$  and  $(30, 0)$ . Also, assume that  $R_i$  of the right most building can be maximum of 1000. That means, the  $L_i$  co-ordinate of left building can be minimum of 1 and  $R_i$  of the right most building can be maximum of 1000.

**Solution:** The most important piece of information is that we know that the left and right coordinates of each and every building are non-negative integers less than 1000. Now why is this important? Because we can assign a height-value to every distinct  $x_i$  coordinate where  $i$  is between 0 and 9,999.

### Algorithm:

- Allocate an array for 1000 elements and initialize all of the elements to 0. Let's call this array *auxHeights*.

- Iterate over all of the buildings and for every  $B_i$  building iterate on the range of  $[l_i.. r_i)$  where  $l_i$  is the left,  $r_i$  is the right coordinate of the building  $B_i$ .
- For every  $x_j$  element of this range check if  $h_i > auxHeights[x_j]$ , that is if building  $B_i$  is taller than the current height-value at position  $x_j$ . If so, replace  $auxHeights[x_j]$  with  $h_i$ .

Once we checked all the buildings, the *auxHeights* array stores the heights of the tallest buildings at every position. There is one more thing to do: convert the *auxHeights* array to the expected output format, that is to a sequence of position-height pairs. It's also easy: just map each and every  $i$  index to an  $(i, auxHeights[i])$  pair.

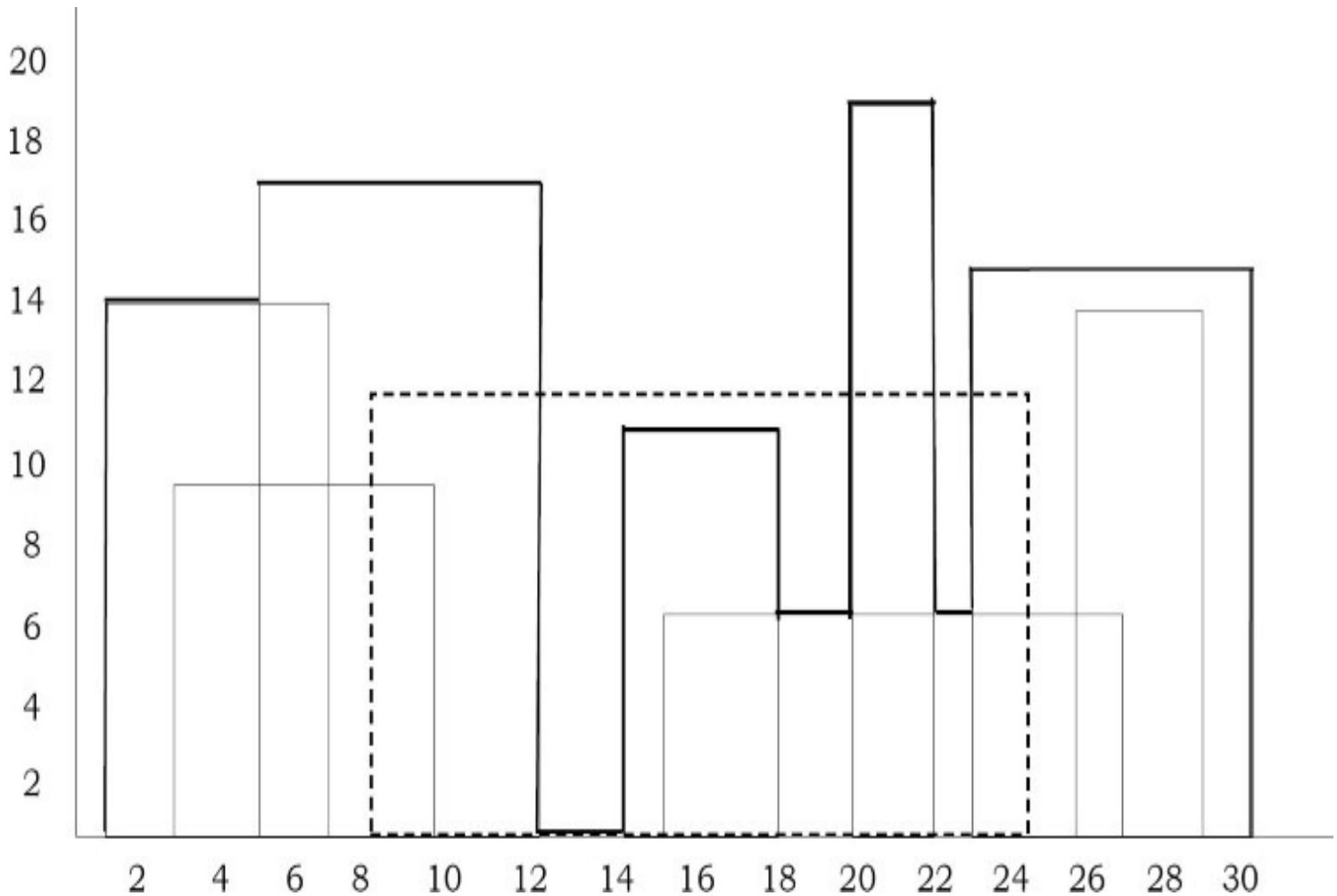
```
#include<stdio.h>
#define MaxRightMostBuildingRi 1000
int auxHeights[MaxRightMostBuildingRi];
int SkyLineBruteForce(){
    int left,h,right,i,prev;
    int rightMostBuildingRi=0;
    while(scanf("%d %d %d", &left, &h, &right)==3){
        for(i=left;i<right;i++){
            if(auxHeights[i]<h)
                auxHeights[i]=h;
            if(rightMostBuildingRi<right)
                rightMostBuildingRi=right;
        }
        prev = 0;
        for(i=1;i<rightMostBuildingRi;i++){
            if(prev!=auxHeights[i]){
                printf("%d %d ", i, auxHeights[i]);
                prev=auxHeights[i];
            }
        }
        printf("%d %d\n", rightMostBuildingRi, auxHeights[rightMostBuildingRi]);
        return 0;
    }
}
```

Let's have a look at the time complexity of this algorithm. Assume that,  $n$  indicates the number of buildings in the input sequence and  $m$  indicates the maximum coordinate (right most building  $r_i$ ). From the above code, it is clear that for every new input building, we are traversing from *left* ( $l_i$ ) to *right* ( $r_i$ ) to update the heights. In the worst case, with  $n$  equal-size buildings, each having  $l = 0$  left and  $r = m - 1$  right coordinates, that is every building spans over the whole  $[0.. m)$  interval.

Thus the running time of setting the height of every position is  $O(n \times m)$ . The overall time-complexity is  $O(n \times m)$ , which is a lot larger than  $O(n^2)$  if  $m > n$ .

**Problem-31** Can we improve the solution of the [Problem-30](#)?

**Solution:** It would be a huge speed-up if somehow we could determine the skyline by calculating the height for those coordinates only where it matters, wouldn't it? Intuition tells us that if we can insert a building into an *existing skyline* then instead of all the coordinates the building spans over we only need to check the height at the left and right coordinates of the building plus those coordinates of the skyline the building overlaps with and may modify.



Is merging two skylines substantially different from merging a building with a skyline? The answer is, of course, No. This suggests that we use divide-and-conquer. Divide the input of  $n$  buildings into two equal sets. Compute (recursively) the skyline for each set then merge the two skylines. Inserting the buildings one after the other is not the fastest way to solve this problem as we've seen it above. If, however, we first merge pairs of buildings into skylines, then we merge pairs of these skylines into bigger skylines (and not two sets of buildings), and then merge pairs of these bigger skylines into even bigger ones, then - since the problem size is halved in every step - after  $\log n$  steps we can compute the final skyline.

```
class SkyLineDivideandConquer {
public:
    vector<pair<int, int>> getSkyline(int start, int end, vector<vector<int>>& buildings) {
        if (start == end) {
            vector<pair<int, int>> result;
            result.push_back(pair<int, int>(buildings[start][0], buildings[start][1]));
            result.push_back(pair<int, int>(buildings[start][2], 0));
            return result;
        }
        int mid = (end + start) / 2;
        vector<pair<int, int>> leftSkyline = getSkyline(start, mid, buildings);
```

```

vector<pair<int, int>> rightSkyline = getSkyline(mid + 1, end, buildings);
vector<pair<int, int>> result = mergeSkylines(leftSkyline, rightSkyline);
return result;
}

vector<pair<int, int>> mergeSkylines(vector<pair<int, int>>& leftSkyline, vector<pair<int, int>>& rightSkyline) {
    vector<pair<int, int>> result;
    int i = 0, j = 0, currentHeight1 = 0, currentHeight2 = 0;
    int maxH = max(currentHeight1, currentHeight2);
    while (i != leftSkyline.size() && j != rightSkyline.size()) {
        if (leftSkyline[i].first < rightSkyline[j].first) {
            currentHeight1 = leftSkyline[i].second;
            if (maxH != max(currentHeight1, currentHeight2))
                result.push_back(pair<int, int>(leftSkyline[i].first, max(currentHeight1, currentHeight2)));
            maxH = max(currentHeight1, currentHeight2);
            i++;
        }
        else if (leftSkyline[i].first > rightSkyline[j].first) {
            currentHeight2 = rightSkyline[j].second;
            if (maxH != max(currentHeight1, currentHeight2))
                result.push_back(pair<int, int>(rightSkyline[j].first, max(currentHeight1, currentHeight2)));
            maxH = max(currentHeight1, currentHeight2);
            j++;
        }
        else {
            if (leftSkyline[i].second >= rightSkyline[j].second) {
                currentHeight1 = leftSkyline[i].second;
                currentHeight2 = rightSkyline[j].second;
                if (maxH != max(currentHeight1, currentHeight2))
                    result.push_back(pair<int, int>(leftSkyline[i].first, leftSkyline[i].second));
                maxH = max(currentHeight1, currentHeight2);
                i++;
                j++;
            }
            else {
                currentHeight1 = leftSkyline[i].second;
                currentHeight2 = rightSkyline[j].second;
                if (maxH != max(currentHeight1, currentHeight2))
                    result.push_back(pair<int, int>(rightSkyline[j].first, rightSkyline[j].second));
                maxH = max(currentHeight1, currentHeight2);
                i++;
                j++;
            }
        }
    }
    while (j < rightSkyline.size()) {
        result.push_back(rightSkyline[j]);
        j++;
    }
    while (i != leftSkyline.size()) {
        result.push_back(leftSkyline[i]);
        i++;
    }
    return result;
}
};

```



For example, given two skylines  $A=(a_1, ha_1, a_2, ha_2, \dots, a_n, 0)$  and  $B=(b_1, hb_1, b_2, hb_2, \dots, b_m, 0)$ , we merge these lists as the new list:  $(c_1, hc_1, c_2, hc_2, \dots, c_{n+m}, 0)$ . Clearly, we merge the list of  $a$ 's and  $b$ 's just like in the standard Merge algorithm. But, in addition to that, we have to decide on the correct height in between these boundary values. We use two variables *currentHeight1* and *currentHeight2* (note that these are the heights prior to encountering the heads of the lists) to store the current height of the first and the second skyline, respectively. When comparing the head entries (*currentHeight1*, *currentHeight2*) of the two skylines, we introduce a new strip (and append to the output skyline) whose x-coordinate is the minimum of the entries' x-coordinates and whose height is the maximum of *currentHeight1* and *currentHeight2*. This algorithm has a structure similar to Mergesort. So the overall running time of the divide and conquer approach will be  $O(n \log n)$ .

---

# DYNAMIC PROGRAMMING

CHAPTER

19



---

## 19.1 Introduction

In this chapter we will try to solve the problems for which we failed to get the optimal solutions using other techniques (say, *Divide & Conquer* and *Greedy* methods). Dynamic Programming (DP) is a simple technique but it can be difficult to master. One easy way to identify and solve DP problems is by solving as many problems as possible. The term *Programming* is not related to coding but it is from literature, and means filling tables (similar to *Linear Programming*).

## 19.2 What is Dynamic Programming Strategy?

Dynamic programming and memoization work together. The main difference between dynamic programming and divide and conquer is that in the case of the latter, sub problems are independent, whereas in DP there can be an overlap of sub problems. By using memoization [maintaining a table of sub problems already solved], dynamic programming reduces the exponential complexity to polynomial complexity ( $O(n^2)$ ,  $O(n^3)$ , etc.) for many problems. The major components of DP are:

- Recursion: Solves sub problems recursively.

- Memoization: Stores already computed values in table (*Memoization* means caching).

*Dynamic Programming = Recursion + Memoization*

### 19.3 Properties of Dynamic Programming Strategy

The two dynamic programming properties which can tell whether it can solve the given problem or not are:

- *Optimal substructure*: an optimal solution to a problem contains optimal solutions to sub problems.
- *Overlapping sub problems*: a recursive solution contains a small number of distinct sub problems repeated many times.

### 19.4 Can Dynamic Programming Solve All Problems?

Like Greedy and Divide and Conquer techniques, DP cannot solve every problem. There are problems which cannot be solved by any algorithmic technique [Greedy, Divide and Conquer and Dynamic Programming].

The difference between Dynamic Programming and straightforward recursion is in memoization of recursive calls. If the sub problems are independent and there is no repetition then memoization does not help, so dynamic programming is not a solution for all problems.

### 19.5 Dynamic Programming Approaches

Basically there are two approaches for solving DP problems:

- Bottom-up dynamic programming
- Top-down dynamic programming

#### Bottom-up Dynamic Programming

In this method, we evaluate the function starting with the smallest possible input argument value and then we step through possible values, slowly increasing the input argument value. While computing the values we store all computed values in a table (memory). As larger arguments are evaluated, pre-computed values for smaller arguments can be used.

#### Top-down Dynamic Programming

In this method, the problem is broken into sub problems; each of these sub problems is solved; and the solutions remembered, in case they need to be solved. Also, we save each computed value as the final action of the recursive function, and as the first action we check if pre-computed value exists.

## Bottom-up versus Top-down Programming

In bottom-up programming, the programmer has to select values to calculate and decide the order of calculation. In this case, all sub problems that might be needed are solved in advance and then used to build up solutions to larger problems. In top-down programming, the recursive structure of the original code is preserved, but unnecessary recalculation is avoided. The problem is broken into sub problems, these sub problems are solved and the solutions remembered, in case they need to be solved again.

**Note:** Some problems can be solved with both the techniques and we will see examples in the next section.

## 19.6 Examples of Dynamic Programming Algorithms

- Many string algorithms including longest common subsequence, longest increasing subsequence, longest common substring, edit distance.
- Algorithms on graphs can be solved efficiently: Bellman-Ford algorithm for finding the shortest distance in a graph, Floyd's All-Pairs shortest path algorithm, etc.
- Chain matrix multiplication
- Subset Sum
- 0/1 Knapsack
- Travelling salesman problem, and many more

## 19.7 Understanding Dynamic Programming

Before going to problems, let us understand how DP works through examples.

### Fibonacci Series

In Fibonacci series, the current number is the sum of previous two numbers. The Fibonacci series is defined as follows:

$$\begin{aligned} \text{Fib}(n) &= 0, & \text{for } n &= 0 \\ &= 1, & \text{for } n &= 1 \\ &= \text{Fib}(n-1) + \text{Fib}(n-2), & \text{for } n &> 1 \end{aligned}$$

The recursive implementation can be given as:

```
int RecursiveFibonacci(int n) {  
    if(n == 0) return 0;  
    if(n == 1) return 1;  
    return RecursiveFibonacci(n - 1) + RecursiveFibonacci(n - 2);  
}
```

Solving the above recurrence gives:

$$T(n) = T(n-1) + T(n-2) + 1 \approx \left(\frac{1+\sqrt{5}}{2}\right)^n \approx 2^n = O(2^n)$$

**Note:** For proof, refer to [Introduction](#) chapter.

### How does Memoization help?

Calling *fib*(5) produces a call tree that calls the function on the same value many times:

*fib*(5)  
*fib*(4) + *fib*(3)  
(*fib*(3) + *fib*(2)) + (*fib*(2) + *fib*(1))  
((*fib*(2) + *fib*(1)) + (*fib*(1) + *fib*(0))) + ((*fib*(1) + *fib*(0)) + *fib*(1))  
(((*fib*(1) + *fib*(0)) + *fib*(1)) + (*fib*(1) + *fib*(0))) + ((*fib*(1) + *fib*(0)) + *fib*(1))

In the above example, *fib*(2) was calculated three times (overlapping of subproblems). If *n* is big, then many more values of *fib* (sub problems) are recalculated, which leads to an exponential time algorithm. Instead of solving the same sub problems again and again we can store the previous calculated values and reduce the complexity.

*Memoization* works like this: Start with a recursive function and add a table that maps the function's parameter values to the results computed by the function. Then if this function is called twice with the same parameters, we simply look up the answer in the table.

**Improving:** Now, we see how DP reduces this problem complexity from exponential to polynomial. As discussed earlier, there are two ways of doing this. One approach is bottom-up: these methods start with lower values of input and keep building the solutions for higher values.

```

int fib[n];
int fib(int n) {
    // Check for base cases
    if(n == 0 || n == 1) return 1;
    fib[0] = 1;
    fib[1] = 1;
    for (int i = 2; i < n; i++)
        fib[i] = fib[i - 1] + fib[i - 2];
    return fib[n - 1];
}

```

The other approach is top-down. In this method, we preserve the recursive calls and use the values if they are already computed. The implementation for this is given as:

```

int fib[n];
int fibonacci( int n ) {
    if(n == 1)
        return 1;
    if(n == 2)
        return 1;
    if( fib[n] != 0) return fib[n] ;
    return fib[n] = fibonacci(n-1) + fibonacci(n -2) ;
}

```

**Note:** For all problems, it may not be possible to find both top-down and bottom-up programming solutions.

Both versions of the Fibonacci series implementations clearly reduce the problem complexity to  $O(n)$ . This is because if a value is already computed then we are not calling the subproblems again. Instead, we are directly taking its value from the table.

Time Complexity:  $O(n)$ . Space Complexity:  $O(n)$ , for table.

**Further Improving:** One more observation from the Fibonacci series is: The current value is the sum of the previous two calculations only. This indicates that we don't have to store all the previous values. Instead, if we store just the last two values, we can calculate the current value. The implementation for this is given below:

```

int fibonacci(int n) {
    int a = 0, b = 1, sum, i;
    for (i=0; i < n; i++) {
        sum = a + b;
        a = b;
        b = sum;
    }
    return sum;
}

```

Time Complexity:  $O(n)$ . Space Complexity:  $O(1)$ .

**Note:** This method may not be applicable (available) for all problems.

## Observations

While solving the problems using DP, try to figure out the following:

- See how the problems are defined in terms of subproblems recursively.
- See if we can use some table [memoization] to avoid the repeated calculations.

## Factorial of a Number

As another example, consider the factorial problem:  $n!$  is the product of all integers between  $n$  and 1. The definition of recursive factorial can be given as:

$$\begin{aligned}
 n! &= n * (n-1)! \\
 1! &= 1 \\
 0! &= 1
 \end{aligned}$$

This definition can easily be converted to implementation. Here the problem is finding the value of  $n!$ , and the sub-problem is finding the value of  $(n-1)!$ . In the recursive case, when  $n$  is greater than 1, the function calls itself to find the value of  $(n-1)!$  and multiplies that with  $n$ . In the base case, when  $n$  is 0 or 1, the function simply returns 1.

```

int fact(int n) {
    if(n == 1) return 1;
    else if(n == 0) return 1;
    else // recursive case: multiply n by (n-1) factorial
        return n * fact(n-1);
}

```

The recurrence for the above implementation can be given as:  $T(n) = n \times T(n - 1) \approx O(n)$   
Time Complexity:  $O(n)$ . Space Complexity:  $O(n)$ , recursive calls need a stack of size  $n$ .

In the above recurrence relation and implementation, for any  $n$  value, there are no repetitive calculations (no overlapping of sub problems) and the factorial function is not getting any benefits with dynamic programming. Now, let us say we want to compute a series of  $m!$  for some arbitrary value  $m$ . Using the above algorithm, for each such call we can compute it in  $O(m)$ . For example, to find both  $n!$  and  $m!$  we can use the above approach, wherein the total complexity for finding  $n!$  and  $m!$  is  $O(m + n)$ .

Time Complexity:  $O(n + m)$ .

Space Complexity:  $O(\max(m, n))$ , recursive calls need a stack of size equal to the maximum of  $m$  and  $n$ .

**Improving:** Now let us see how DP reduces the complexity. From the above recursive definition it can be seen that  $fact(n)$  is calculated from  $fact(n - 1)$  and  $n$  and nothing else. Instead of calling  $fact(n)$  every time, we can store the previous calculated values in a table and use these values to calculate a new value. This implementation can be given as:

```
int facto[n];
int fact(int n) {
    if(n == 1) return 1;
    else if(n == 0)
        return 1;
    //Already calculated case
    else if(facto[n] != 0)
        return facto[n];
    else // recursive case: multiply n by (n - 1) factorial
        return facto[n] = n * fact(n - 1);
}
```

For simplicity, let us assume that we have already calculated  $n!$  and want to find  $m!$ . For finding  $m!$ , we just need to see the table and use the existing entries if they are already computed. If  $m < n$  then we do not have to recalculate  $m!$ . If  $m > n$  then we can use  $n!$  and call the factorial on the remaining numbers only.

The above implementation clearly reduces the complexity to  $O(\max(m, n))$ . This is because if the  $fact(n)$  is already there, then we are not recalculating the value again. If we fill these newly computed values, then the subsequent calls further reduce the complexity.

Time Complexity:  $O(\max(m, n))$ . Space Complexity:  $O(\max(m, n))$  for table.

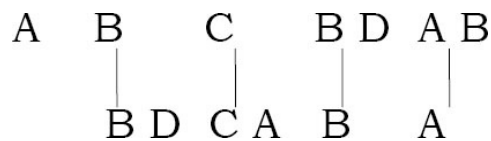


## 19.8 Longest Common Subsequence

Given two strings: string  $X$  of length  $m$  [ $X(1..m)$ ], and string  $Y$  of length  $n$  [ $Y(1..n)$ ], find the longest common subsequence: the longest sequence of characters that appear left-to-right (but not necessarily in a contiguous block) in both strings. For example, if  $X = \text{"ABCBBDAB"}$  and  $Y = \text{"BDCABA"}$ , the  $LCS(X, Y) = \{\text{"BCBA"}, \text{"BDAB"}, \text{"BCAB"}\}$ . We can see there are several optimal solutions.

**Brute Force Approach:** One simple idea is to check every subsequence of  $X[1..m]$  ( $m$  is the length of sequence  $X$ ) to see if it is also a subsequence of  $Y[1..n]$  ( $n$  is the length of sequence  $Y$ ). Checking takes  $O(n)$  time, and there are  $2^m$  subsequences of  $X$ . The running time thus is exponential  $O(n \cdot 2^m)$  and is not good for large sequences.

**Recursive Solution:** Before going to DP solution, let us form the recursive solution for this and later we can add memoization to reduce the complexity. Let's start with some simple observations about the LCS problem. If we have two strings, say  $\text{"ABCBBDAB"}$  and  $\text{"BDCABA"}$ , and if we draw lines from the letters in the first string to the corresponding letters in the second, no two lines cross:



From the above observation, we can see that the current characters of  $X$  and  $Y$  may or may not match. That means, suppose that the two first characters differ. Then it is not possible for both of them to be part of a common subsequence - one or the other (or maybe both) will have to be removed. Finally, observe that once we have decided what to do with the first characters of the strings, the remaining sub problem is again a  $LCS$  problem, on two shorter strings. Therefore we can solve it recursively.

The solution to  $LCS$  should find two sequences in  $X$  and  $Y$  and let us say the starting index of sequence in  $X$  is  $i$  and the starting index of sequence in  $Y$  is  $j$ . Also, assume that  $X[i \dots m]$  is a substring of  $X$  starting at character  $i$  and going until the end of  $X$ , and that  $Y[j \dots n]$  is a substring of  $Y$  starting at character  $j$  and going until the end of  $Y$ .

Based on the above discussion, here we get the possibilities as described below:

- 1) If  $X[i] == Y[j]$  :  $1 + LCS(i + 1, j + 1)$
- 2) If  $X[i] \neq Y[j]$ .  $LCS(i, j + 1)$  // skipping  $j^{th}$  character of  $Y$
- 3) If  $X[i] \neq Y[j]$ .  $LCS(i + 1, j)$  // skipping  $i^{th}$  character of  $X$

In the first case, if  $X[i]$  is equal to  $Y[j]$ , we get a matching pair and can count it towards the total length of the  $LCS$ . Otherwise, we need to skip either  $i^{th}$  character of  $X$  or  $j^{th}$  character of  $Y$  and find the longest common subsequence. Now,  $LCS(i, j)$  can be defined as:

$$LCS(i, j) = \begin{cases} 0, & \text{if } i = m \text{ or } j = n \\ \text{Max}\{LCS(i, j + 1), LCS(i + 1, j)\}, & \text{if } X[i] \neq Y[j] \\ 1 + LCS[i + 1, j + 1], & \text{if } X[i] == Y[j] \end{cases}$$

LCS has many applications. In web searching, if we find the smallest number of changes that are needed to change one word into another. A *change* here is an insertion, deletion or replacement of a single character.

```
//Initial Call: LCSLength(X, 0, m-1, Y, 0, n-1);
int LCSLength( int X[], int i, int m, int Y[], int j, int n) {
    if (i == m || j == n)
        return 0;
    else if (X[i] == Y[j]) return 1 + LCSLength(X, i+1, m, Y, j+1, n);
    else return max( LCSLength(X, i+1, m, Y, j, n), LCSLength(X, i, m, Y, j+1, n));
}
```

This is a correct solution but it is very time consuming. For example, if the two strings have no matching characters, the last line always gets executed which gives (if  $m == n$ ) close to  $O(2^n)$ .

**DP Solution: Adding Memoization:** The problem with the recursive solution is that the same subproblems get called many different times. A subproblem consists of a call to `LCS_length`, with the arguments being two suffixes of  $X$  and  $Y$ , so there are exactly  $(i + 1)(j + 1)$  possible subproblems (a relatively small number). If there are nearly  $2^n$  recursive calls, some of these subproblems must be being solved over and over.

The DP solution is to check, whenever we want to solve a sub problem, whether we've already done it before. So we look up the solution instead of solving it again. Implemented in the most direct way, we just add some code to our recursive solution. To do this, look up the code. This can be given as:



Space Complexity:  $O(mn)$ .

**Note:** In the above discussion, we have assumed  $LCS(i,j)$  is the length of the  $LCS$  with  $X[i \dots m]$  and  $Y[j \dots n]$ . We can solve the problem by changing the definition as  $LCS(i,j)$  is the length of the  $LCS$  with  $X[1 \dots i]$  and  $Y[1 \dots j]$ .

**Printing the subsequence:** The above algorithm can find the length of the longest common subsequence but cannot give the actual longest subsequence. To get the sequence, we trace it through the table. Start at cell  $(0,0)$ . We know that the value of  $LC5[0][0]$  was the maximum of 3 values of the neighboring cells. So we simply recompute  $LC5[0][0]$  and note which cell gave the maximum value. Then we move to that cell (it will be one of  $(1,1)$ ,  $(0,1)$  or  $(1,0)$ ) and repeat this until we hit the boundary of the table. Every time we pass through a cell  $(i,j')$  where  $X[i] == Y[j]$ , we have a matching pair and print  $X[i]$ . At the end, we will have printed the longest common subsequence in  $O(mn)$  time.

An alternative way of getting path is to keep a separate table for each cell. This will tell us which direction we came from when computing the value of that cell. At the end, we again start at cell  $(0,0)$  and follow these directions until the opposite corner of the table.

From the above examples, I hope you understood the idea behind DP. Now let us see more problems which can be easily solved using the DP technique.

**Note:** As we have seen above, in DP the main component is recursion. If we know the recurrence then converting that to code is a minimal task. For the problems below, we concentrate on getting the recurrence.

## 19.9 Dynamic Programming: Problems & Solutions

**Problem-1** Convert the following recurrence to code.

$$T(0) = T(1) = 2$$
$$T(n) = \sum_{i=1}^{n-1} 2 \times T(i) \times T(i-1), \text{ for } n > 1$$

**Solution:** The code for the given recursive formula can be given as:

```

int f(int n) {
    int sum = 0;
    if(n==0 || n==1) //Base Case
        return 2;
    //recursive case
    for(int i=1; i < n; i++)
        sum += 2 * f(i) * f(i-1);
    return sum;
}

```

**Problem-2** Can we improve the solution to [Problem-1](#) using memoization of DP?

**Solution: Yes.** Before finding a solution, let us see how the values are calculated.

$$T(0) = T(1) = 2$$

$$T(2) = 2 * T(1) * T(0)$$

$$T(3) = 2 * T(1) * T(0) + 2 * T(2) * T(1)$$

$$T(4) = 2 * T(1) * T(0) + 2 * T(2) * T(1) + 2 * T(3) * T(2)$$

From the above calculations it is clear that there are lots of repeated calculations with the same input values. Let us use a table for avoiding these repeated calculations, and the implementation can be given as:

```

int f(int n) {
    T[0] = T[1] = 2;
    for(int i=2; i <= n; i++) {
        T[i] = 0;
        for (int j=1; j < i; j++)
            T[i] += 2 * T[j] * T[j-1];
    }
    return T[n];
}

```

Time Complexity:  $O(n^2)$ , two *for* loops. Space Complexity:  $O(n)$ , for table.

**Problem-3** Can we further improve the complexity of [Problem-2](#)?

**Solution: Yes,** since all sub problem calculations are dependent only on previous calculations, code can be modified as:

```

int f(int n) {
    T[0] = T[1] = 2;
    T[2] = 2 * T[0] * T[1];
    for(int i=3; i <= n; i++)
        T[i] = T[i-1] + 2 * T[i-1] * T[i-2];
    return T[n];
}

```

Time Complexity:  $O(n)$ , since only one *for* loop. Space Complexity:  $O(n)$ .

**Problem-4 Maximum Value Contiguous Subsequence:** Given an array of  $n$  numbers, give an algorithm for finding a contiguous subsequence  $A(i) \dots A(j)$  for which the sum of elements is maximum. **Example:**  $\{-2, 11, -4, 13, -5, 2\} \rightarrow 20$  and  $\{1, -3, 4, -2, -1, 6\} \rightarrow 7$

**Solution:**

**Input:** Array.  $A(1) \dots A(n)$  of  $n$  numbers.

**Goal:** If there are no negative numbers, then the solution is just the sum of all elements in the given array. If negative numbers are there, then our aim is to maximize the sum [there can be a negative number in the contiguous sum].

One simple and brute force approach is to see all possible sums and select the one which has maximum value.

```

int MaxContiguousSum(int A[], in n) {
    int maxSum = 0;
    for(int i = 0; i < n; i++)                // for each possible start point
        for(int j = i; j < n; j++)            // for each possible end point
            {
                int currentSum = 0;
                for(int k = i; k <= j; k++)
                    currentSum += A[k];
                if(currentSum > maxSum)
                    maxSum = currentSum;
            }
    }
    return maxSum;
}

```

Time Complexity:  $O(n^3)$ . Space Complexity:  $O(1)$ .

**Problem-5** Can we improve the complexity of [Problem-4](#)?

**Solution: Yes.** One important observation is that, if we have already calculated the sum for the subsequence  $i, \dots, j - 1$ , then we need only one more addition to get the sum for the subsequence  $i, \dots, j$ . But, the [Problem-4](#) algorithm ignores this information. If we use this fact, we can get an improved algorithm with the running time  $O(n^2)$ .

```
int MaxContiguousSum(int A[], int n) {
    int maxSum = 0;
    for( int i = 0; i < n; i++) {
        int currentSum = 0;
        for( int j = i; j < n; j++) {
            currentSum += a[j];
            if(currentSum > maxSum)
                maxSum = currentSum;
        }
    }
    return maxSum;
}
```

Time Complexity:  $O(n^2)$ . Space Complexity:  $O(1)$ .

**Problem-6** Can we solve [Problem-4](#) using Dynamic Programming?

**Solution: Yes.** For simplicity, let us say,  $M(i)$  indicates maximum sum over all windows ending at  $i$ .

Given Array, A: recursive formula considers the case of selecting  $i^{th}$  element

|  |       |   |  |
|--|-------|---|--|
|  | ..... | ? |  |
|--|-------|---|--|

$A[i]$

To find maximum sum we have to do one of the following and select maximum among them.

- Either extend the old sum by adding  $A[i]$
- or start new window starting with one element  $A[i]$

$$M(i) = \text{Max} \begin{cases} M(i - 1) + A[i] \\ 0 \end{cases}$$

Where,  $M(i - 1) + A[i]$  indicates the case of extending the previous sum by adding  $A[i]$  and 0 indicates the new window starting at  $A[i]$ .

```

int MaxContiguousSum(int A[], int n) {
    int M[n] = 0, maxSum = 0;
    if(A[0] > 0)
        M[0] = A[0];
    else M[0] = 0;
    for( int i = 1; i < n; i++) {
        if( M[i-1] + A[i] > 0)
            M[i] = M[i-1] + A[i];
        else    M[i] = 0;
    }
    for( int i = 0; i < n; i++)
        if(M[i] > maxSum)
            maxSum = M[i];
    return maxSum;
}

```

Time Complexity:  $O(n)$ . Space Complexity:  $O(n)$ , for table.

**Problem-7** Is there any other way of solving [Problem-4](#)?

**Solution: Yes.** We can solve this problem without DP too (without memory). The algorithm is a little tricky. One simple way is to look for all positive contiguous segments of the array (*sumEndingHere*) and keep track of the maximum sum contiguous segment among all positive segments (*sumSoFar*). Each time we get a positive sum compare it (*sumEndingHere*) with *sumSoFar* and update *sumSoFar* if it is greater than *sumSoFar*. Let us consider the following code for the above observation.

```

int MaxContiguousSum(int A[], int n) {
    int sumSoFar = 0, sumEndingHere = 0;
    for(int i = 0; i < n; i++) {
        sumEndingHere = sumEndingHere + A[i];
        if(sumEndingHere < 0) {
            sumEndingHere = 0;
            continue;
        }
        if(sumSoFar < sumEndingHere)
            sumSoFar = sumEndingHere;
    }
    return sumSoFar;
}

```



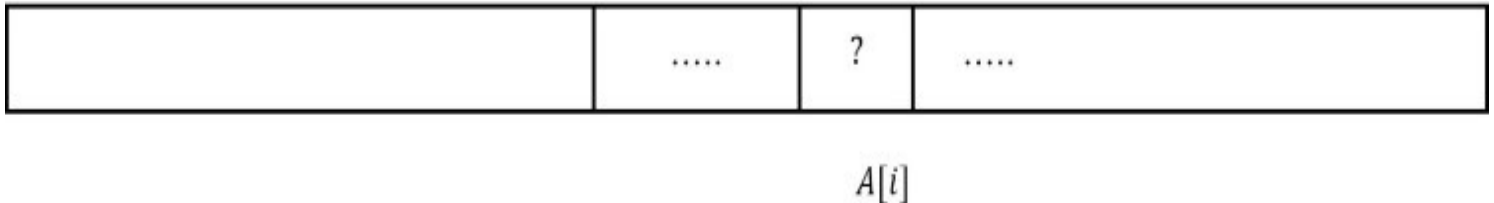
**Note:** The algorithm doesn't work if the input contains all negative numbers. It returns 0 if all numbers are negative. To overcome this, we can add an extra check before the actual implementation. The phase will look if all numbers are negative, and if they are it will return maximum of them (or smallest in terms of absolute value).

Time Complexity:  $O(n)$ , because we are doing only one scan. Space Complexity:  $O(1)$ , for table.

**Problem-8** In [Problem-7](#) solution, we have assumed that  $M(i)$  indicates maximum sum over all windows ending at  $i$ . Can we assume  $M(i)$  indicates maximum sum over all windows starting at  $i$  and ending at  $n$ ?

**Solution:** Yes. For simplicity, let us say,  $M(i)$  indicates maximum sum over all windows starting at  $i$ .

Given Array, A: recursive formula considers the case of selecting  $i^{th}$  element



To find maximum window we have to do one of the following and select maximum among them.

- Either extend the old sum by adding  $A[i]$
- Or start new window starting with one element  $A[i]$

$$M(i) = \text{Max} \begin{cases} M(i+1) + A[i], & \text{if } M(i+1) + A[i] > 0 \\ 0, & \text{if } M(i+1) + A[i] \leq 0 \end{cases}$$

Where,  $M(i+1) + A[i]$  indicates the case of extending the previous sum by adding  $A[i]$ , and 0 indicates the new window starting at  $A[i]$ .

Time Complexity:  $O(n)$ . Space Complexity:  $O(n)$ , for table.

**Note:** For  $O(n \log n)$  solution, refer to the [Divide and Conquer](#) chapter.

**Problem-9** Given a sequence of  $n$  numbers  $A(1) \dots A(n)$ , give an algorithm for finding a contiguous subsequence  $A(i) \dots A(j)$  for which the sum of elements in the subsequence is maximum. Here the condition is we should not select *two* contiguous numbers.

**Solution:** Let us see how DP solves this problem. Assume that  $M(i)$  represents the maximum sum from 1 to  $i$  numbers without selecting two contiguous numbers. While computing  $M(i)$ , the decision we have to make is, whether to select the  $i^{th}$  element or not. This gives us two possibilities and based on this we can write the recursive formula as:

$$M(i) = \begin{cases} \text{Max}\{A[i] + M(i - 2), M(i - 1)\}, & \text{if } i > 2 \\ A[1], & \text{if } i = 1 \\ \text{Max}\{A[1], A[2]\}, & \text{if } i = 2 \end{cases}$$

- The first case indicates whether we are selecting the  $i^{\text{th}}$  element or not. If we don't select the  $i^{\text{th}}$  element then we have to maximize the sum using the elements 1 to  $i - 1$ . If  $i^{\text{th}}$  element is selected then we should not select  $i - 1^{\text{th}}$  element and need to maximize the sum using 1 to  $i - 2$  elements.
- In the above representation, the last two cases indicate the base cases.

Given Array, A: recursive formula considers the case of selecting  $i^{\text{th}}$  element

|  |       |       |   |  |
|--|-------|-------|---|--|
|  | ..... | ..... | ? |  |
|--|-------|-------|---|--|

$A[i-2]$      $A[i-1]$      $A[i]$

```
int maxSumWithNoTwoContinuousNumbers(int A[], int n) {
    int M[n+1];
    M[0]=A[0];
    M[1]=(A[0]>A[1]?A[0]:A[1]);
    for(i=2, i<n; i++)
        M[i]= (M[i-1]>M[i-2]+A[i]? M[i-1]: M[i-2]+A[i]);
    return M[n-1];
}
```

Time Complexity:  $O(n)$ . Space Complexity:  $O(n)$ .

**Problem-10** In [Problem-9](#), we assumed that  $M(i)$  represents the maximum sum from 1 to  $i$  numbers without selecting two contiguous numbers. Can we solve the same problem by changing the definition as:  $M(i)$  represents the maximum sum from  $i$  to  $n$  numbers without selecting two contiguous numbers?

**Solution: Yes.** Let us assume that  $M(i)$  represents the maximum sum from  $i$  to  $n$  numbers without selecting two contiguous numbers:

Given Array, A: recursive formula considers the case of selecting  $i^{\text{th}}$  element

|  |   |       |       |  |
|--|---|-------|-------|--|
|  | ? | ..... | ..... |  |
|--|---|-------|-------|--|

$A[i]$      $A[i+1]$      $A[i+2]$